

基于贪心思想的沙漠穿越策略

【摘要】

本文主要对玩家穿越沙漠的采取策略进行设计，根据天气状况、资源消耗、挖矿收益、剩余资金、水量和食物量等因素，以到达终点后保留资金数最多为目标，建立最优化模型，并设计算法，利用 *Matlab* 编程求解。

针对问题一，在每天天气状况已知的情况下，规划一般情况下玩家的最优策略。根据玩家在每天不同天气的资源消耗，判断何时需要购买水和食物，何时需要挖矿。建立邻接矩阵进行路线规划，对给定初始水量和食物量，玩家行走方案，在不同的组合搭配下搜索最优解。第一关：求解得玩家第 0 天在起点购买水 180 箱、食物 330 箱时，最终剩余资金数最多为 10430 元。第二关：求解得玩家第 0 天在起点购买水 130 箱、食物 405 箱，最终所剩余资金最多为 12730 元，最优策略的相应具体结果详见支撑材料 *Result_1.xlsx*。

针对问题二，在仅知道当天天气情况下，根据当天天气状况规划玩家的行动策略。结合蒙特卡罗随机法模拟出一般天气情况、极端高温天气与极端晴朗天气情况分别考虑。第三关：在极端高温天气的玩家剩余总资金数为 9190 元；一般天气情况下玩家剩余总资金 9350 元；极端晴朗天气玩家剩余总资金 9430 元，具体策略的结果见正文。第四关：极端高温剩余总资金数为 8787.5 元，一般天气剩余总资金数为 12622.5 元，极端晴朗剩余总资金数为 20012.5 元，且最佳策略的相应具体结果详见支撑材料 *Result_2.xlsx*。

针对问题三，对多名玩家的情况，消耗资源和资金的计算规则发生变化。第五关：每天天气状况已知，沿用问题一的思路，更改目标为所有玩家总资金数最大。通过分析各玩家是否经过矿山、中途是否相遇等设计三个方案。方案一，玩家不挖矿会相遇时，剩余总资金 18800 元。方案二，玩家挖矿，但不相遇，剩余总资金 18175 元。方案三，玩家不挖矿不相遇时，剩余总资金 18910 元。其采取的最佳策略的相应具体结果详见支撑材料 *Result_3_1.xlsx*。第六关：结合第四关与第五关的优化模型，研究 3 名玩家仅知道当天天气的情况下，对一般天气情况进行求解，第 1 名玩家在第 23 天到达终点后剩余资金约为 8748.33 元，第 2 名玩家在第 20 天到达终点后剩余资金约为 6524.17 元，第 3 名玩家在第 18 天到达终点后剩余资金约为 7274.17 元，所有玩家总剩余为 22546.67 元，其采取的最佳策略的相应具体结果详见支撑材料 *Result_3_2.xlsx*。

关键词： 蒙特卡罗随机法 贪心思想 邻接矩阵 最优化模型 *Matlab* 编程

一、问题重述

1.1 引言

穿越沙漠是著名设计师 Reiner Knizia 设计的一款需要策略和耐心的游戏。在穿越沙漠中，玩家凭借一张地图，利用初始资金购买一定数量的水和食物（包括食品和其他日常用品），从起点出发，在沙漠中行走。途中会遇到不同的天气，也可在矿山、村庄补充资金或资源，目标是在规定时间内到达终点，并保留尽可能多的资金。

在玩家穿越沙漠的最优策略中，需考虑诸多因素，如每天天气状况、资源消耗量、挖矿的收益、剩余资金、水量和食物量等。根据本题给出的游戏基本规则，不同关卡，不同的天气状况、资金或地图路线等在策略上均不同，相关参数设定见附件。

1.2 问题提出

请根据游戏的不同设定，结合游戏基本规则和附件中给出的相关数据，建立数学模型，研究以下问题：

1. 假设只有一名玩家，在整个游戏时段内每天天气状况事先全部已知，试给出一般情况下玩家的最优策略。求解附件中的“第一关”和“第二关”，并将相应结果分别填入 Result_1.xlsx。

2. 假设只有一名玩家，玩家仅知道当天的天气状况，可据此决定当天的行动方案，试给出一般情况下玩家的最佳策略，并对附件中的“第三关”和“第四关”进行具体讨论。

3. 现有 n 名玩家，他们有相同的初始资金，且同时从起点出发。若某天其中的任意 k ($2 \leq k \leq n$) 名玩家行走同一条线路、在同一个矿山和同时在一个村庄购买资源时，则他们中的任一位消耗的资源数量、获得的资金和购买资源的价格规则均发生改变，其他情况下消耗资源数量与资源价格与单人游戏相同。

(1) 假设在整个游戏时段内每天天气状况事先全部已知，每名玩家的行动方案需在第 0 天确定且此后不能更改。试给出一般情况下玩家应采取的策略，并对附件中的“第五关”进行具体讨论。

(2) 假设所有玩家仅知道当天的天气状况，从第 1 天起，每名玩家在当天行动结束后均知道其余玩家当天的行动方案和剩余的资源数量，随后确定各自第二天的行动方案。试给出一般情况下玩家应采取的策略，并对附件中的“第六关”进行具体讨论。

二、问题分析

本题主要是对玩家在不同情况下穿越沙漠的最优策略进行研究。针对问题 1，需给出单人玩家已知整个游戏时段内每天天气状况的情况下的最优策略；针对问题 2，需给出单人玩家仅知道当天的天气状况的情况下的最优策略；针对问题 3，需在问题 1 和问题 2 的基础上，给出多名玩家在天气状况已知和未知情况下的最优策略。结合以上三个具体问题，分别做如下分析：

2.1 问题 1 分析

在第一问中，需要考虑单人游戏中已知整个游戏时段内每天天气状况事先全部已知的情况下，求解附件中的“第一关”和“第二关”的玩家最优策略，设计玩家的最优策略的具体求解算法，需要着重从以下几个方面进行分析：

①为了满足在规定时间内到达终点，需要规划玩家在任意一个区域到达终点的最短路线，且保证玩家到该区域后，必须在截止日期之前可以到达终点。采用邻接矩阵表示附件所给地图中区域之间的关系，建立“0-1”变量使玩家行走路线要满足邻接矩阵的相连通关系，确定任意两个不相邻区域的最短路径。

②在满足游戏的基本规则的前提下，使玩家到达终点时保留尽可能多的资金。根据玩家在不同天气状况的资源消耗，判断何时需要在村庄购买水和食物，判断何时需要挖矿增加资金，保证游戏不会失败。以规定时间内到达终点后所保留的资金最多为目标，建立最优化模型 I，设计算法分别求解附件中的“第一关”和“第二关”的一般情况下玩家的最优策略。

2.2 问题 2 分析

在第二问中，需要在问题 1 的模型基础上，考虑单人游戏中玩家仅知道当天的天气状况，从而决定当天的行动方案，求解附件中的“第三关”和“第四关”玩家的最佳策略，针对这两关，还需要分开两部分分析：

①在第三关中，已知 10 天内不会出现沙暴天气，则不需要考虑整个游戏时段内出现沙暴日原地停留的情况；且不需要考虑玩家的水和食物出现不足时，第二次购买水和食物情况，在整个过程中，第 0 天后资金也不会出现减少的情况。

②在第四关中，已知 30 天内较少出现沙暴天气，假设可能出现的沙暴概率为 1~10%，采用蒙特卡洛随机模拟出现沙暴的日期，分别研究 30 天内大部分是高温、大部分是晴朗和高温和晴朗随机均匀的一般情况的三种不同天气状况方案，对三种方案下的一般情况下玩家的最佳策略进行求解，以总资金最多为目标，建立最优化模型 II，设计具体求解算法，利用编程求解相对应的具体结果。

2.3 问题 3 分析

在第三问中，需要研究两个小问，分别主要在问题 1 和问题 2 的基础上，需要考虑多名玩家时对游戏规则的影响，从而确定一般情况下玩家应采取的策略，针对两个小问的具体分析如下：

①第 1 小问主要是在每天天气状况事先已知的情况，对每天每个区域的“第五关”的两名玩家应当做出的最优策略进行研究。由于当两名玩家在同一路线上行进，或某天在同一村庄内购买物资时，每名玩家的资源或资金消耗均会增大，考虑在让对每名玩家规划线路时，尽量避免两名玩家在某天同个地区做出相同行为。

②第 2 小问需要在玩家数量为 3 人时，针对“第六关”的地图信息以及初始参数。在每名玩家仅知道当前天气与其他玩家的行动方案的情况下，研究令 3 名玩家抵达终点时的剩余资金总和最大的行进策略。考虑采取蒙特卡洛随机算法来模拟生成天气情况，而后每名玩家在进行下一步行动前，优先根据其他玩家的行动方案，错开自身与其他玩家的行动时间，减少相同行走或同时购买带来的损失。以所有玩家剩余资金数最大为目标，建立优化模型，设计求解算法，编程所有玩家的最佳策略进行求解。

三、符号说明

符号	描述说明
Y_0	附件中玩家的初始资金
f_{ij}	第 i 天是否停留在地图上的第 j 个区域
g_{ij}	第 i 天是否会到地图上的第 j 个区域
r_i	第 i 天是否会在矿山挖矿
y_i	第 i 天剩余的资金
a_i	第 i 天剩余的水量
b_i	第 i 天剩余的食物量
u_i	第 i 天水的基础消耗量
v_i	第 i 天食物的基础消耗量
$D_{n \times n}$	区域与区域之间的邻接矩阵
h_i	第 i 天消耗的资源数量为基础消耗量的倍数
w_i	第 i 天的剩余水量、食物量、资金的函数关系
s_{ij}	第 i 天经过第 j 个区域的玩家人数
q_{ij}	第 i 天在第 j 个区域购买资源的价格倍数

四、模型假设

- 1、假设游戏中的村庄物资量储备不会耗尽。
- 2、假设游戏中不考虑玩家的实际身体因素。
- 3、假设游戏中玩家携带的物资不会出现遗失情况。
- 4、假设游戏中玩家的移动时间不会出现偏差。

五、模型建立与求解

针对本题游戏的不同设定，要给出一般情况下玩家的最优策略，以规定时间内到达终点为约束，以到达终点保留的资金最多为目标，建立最优化模型，设计相对应的算法，得出玩家的最优策略。

5.1 问题 1：每天天气状况事先已知的最优策略

在本问中，假设是单人游戏，在整个游戏时段内每天天气状况事先全部已知，见附件中的参数设定和地图，以规定时间内到达终点后所保留的资金最多为目标，建立最优化模型，设计算法，分别求解附件中的“第一关”和“第二关”的一般情况下玩家的最优策略。

5.1.1 一般情况下玩家的最佳策略

根据附件中给定的参数设定、天气状况和地图，给出一般情况下玩家的最优策略。由于沙漠中所有区域的天气相同，且每天的天气为“晴朗”、“高温”、“沙暴”三种状况之一，三种状况的基础消耗量均不同，则需要对每天已知的天气状况进行分类和给出所给地图中的任意两个区域的邻接矩阵。

(1) 不同天气状况的集合

根据附件中第一关和第二关的参数设定、天气状况和地图，可以得知第一关、第二关的参数设定和天气状况均相同，则根据附件中的天气状况可以得到沙暴日、晴朗日和高温日相对应的日期集合，即集合可表示如下：

$$\text{沙暴日: } C = \{4, 7, 11, 17, 18, 25\}$$

$$\text{晴朗日: } C' = \{3, 5, 8, 13, 21, 22, 24, 27, 28\}$$

$$\text{高温日: } C'' = \{1, 2, 6, 9, 10, 12, 14, 15, 16, 19, 20, 23, 26, 29, 30\}$$

(2) 地图中的某个区域到达与之相邻的另一个区域的邻接矩阵

结合题意，在所给地图中，有公共边界的两个区域称为相邻，仅有公共顶点而没有公共边界的两个区域不视作相邻，考虑采用邻接矩阵表示任意两个区域的关系。在图论中，邻接矩阵是表示顶点之间相邻关系的矩阵，是存放顶点间关系（边或弧）的数据，用一个顺序表来存储顶点信息。设 $G = (V, E)$ 是具有 n 个顶

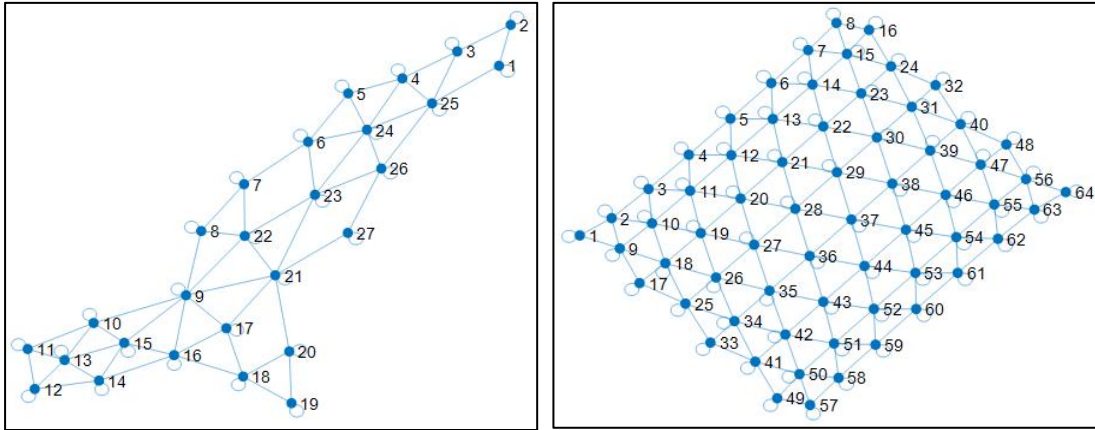
点的图，则地图的邻接矩阵是具有如下性质的 n 阶方阵^[1]：

$$D[i,j]=\begin{cases} 1 & \text{若}(v_i,v_j)\text{或}\langle v_i,v_j \rangle\text{是}E(G)\text{中的边} \\ 0 & \text{若}(v_i,v_j)\text{或}\langle v_i,v_j \rangle\text{不是}E(G)\text{中的边} \end{cases}$$

结合附件所给的地图，设附件中第一关的邻接矩阵 D 是 $n \times n$ 的方阵，有公共边界的两个区域称为相邻，其权值为1，否则权值为0，则可以得到地图中的任意区域到达另一个区域的邻接矩阵 D 表示为：

$$D_{n \times n} = \begin{bmatrix} 1 & 1 & \cdots & 0 & 0 \\ 1 & 1 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & 1 & 1 \\ 0 & 0 & \cdots & 1 & 1 \end{bmatrix}$$

其中，以上邻接矩阵 D 的具体数据见支撑材料附件1的第一关。由于任意两个区域可以来回多次经过，则可以确定所给地图是属于无向图，而无向图的邻接矩阵是对称矩阵；同理，也可以直接得到第二关邻接矩阵，其具体数据见支撑材料附件1的第二关。即第一关和第二关的无向连通图如下图1所示：



(a) 第一关

(b) 第二关

图 1：无向连通图

5.1.2 确定目标

为了确定在规定时间内到达终点，并保留尽可能多的资金，设玩家第0天初始资金为 Y_0 ，在起点购买资源花费的资金为 Y_1 ，在矿山挖矿获得资金为 Y_3 ，在村庄购买的水和食物花费的资金为 Y_2 ，到达终点后退回剩余的水和食物得到的资金为 Y_t ，则以规定时间内到达终点后剩余的总资金最多为目标，可确定目标如下：

$$\text{Max } z = Y_0 - Y_1 + Y_3 - Y_2 + Y_t$$

5.1.3 约束条件

为了满足本题给出游戏的基本规则，需对各个条件做如下约束：

(1) 玩家到达终点的时间约束

根据游戏规则（1）的要求，玩家必须在截止日期或之前达到终点，到达终点后该玩家的游戏结束，否则游戏失败。则需要对玩家达到终点的日期 t 不得小于附件中给出的截止日期 $T = 30$ 天，具体约束如下：

$$t \leq T$$

(2) 玩家拥有的水和食物质量的约束

根据游戏基本规则（2），若未到达终点而水或食物已耗尽，视为游戏失败，则玩家每天拥有的水或食物均不能为0，设玩家每天拥有的水量（箱）为 a_i ，食物量（箱）为 b_i ，则玩家每天拥有的水量和食物量必须满足取自自然数集，具体约束如下：

$$\begin{cases} a_i \in N^* & i = 1, 2, \dots, t-1 \\ b_i \in N^* & i = 1, 2, \dots, t-1 \end{cases}$$

由于每天玩家拥有的水和食物质量之和不能超过负重上限 $M = 1200\text{kg}$ ，且未到达终点时，水或食物均不能耗尽，设每箱水的质量（kg）为 m_1 ，每箱食物的质量（kg）为 m_2 ，则玩家每天拥有的水和质量之和必须满足大于0，且必须满足不能超过负重上限，具体约束如下：

$$0 \leq a_i m_1 + b_i m_2 \leq M \quad i = 0, 1, \dots, t$$

其中，以上的约束条件中， a_0 和 b_0 表示玩家第0天在起点处购买的水和食物的数量（箱）。

(3) “0-1”变量的约束

考虑到玩家行走一天消耗的资源数量是原地停留一天消耗资源数量的2倍，则需要对第 i 天是否在地图上的第 j 个区域原地停留 f_{ij} 进行“0-1”变量约束，具体约束如下：

$$f_{ij} \in \{0, 1\} \quad i = 1, 2, \dots, t; j = 1, 2, \dots, n$$

考虑到附件所给地图中，有公共边界的两个区域称为相邻，仅有公共顶点而没有公共边界的两个区域不视作相邻。玩家每天可从地图中的某个区域到达与之相邻的另一个区域，则需对第 i 天是否到地图上第 j 个区域 g_{ij} 进行“0-1”变量约束，具体约束如下：

$$g_{ij} \in \{0,1\} \quad i=1,2,\dots,t; j=1,2,\dots,n$$

由于玩家在矿山停留时，可以挖矿和原地停留两种情况，则需要对第 i 天是否挖矿 r_i 进行“0-1”变量约束，具体约束如下：

$$r_i \in \{0,1\} \quad i=1,2,\dots,t$$

(4) 两个变量之间的约束

根据以上的两个变量的约束，①当 $f_{ij} = 1$ 时，表示第 i 天停留在第 j 个区域，则第 i 天一定到在第 j 个区域，即 $g_{ij} = 1$ ；②当 $f_{ij} = 0$ 时，表示第 i 天没有停留在第 j 个区域，则第 i 天在第 j 个区域时，即 $g_{ij} = 1$ ，③否则没有经过该区域，即 $g_{ij} = 0$ ，所以可以得到第 i 天是否在地图上的第 j 个区域原地停留 f_{ij} 与第 i 天是否到第 j 个区域 g_{ij} 之间的约束，具体约束如下：

$$f_{ij} \leq g_{ij} \quad i=1,2,\dots,t; j=1,2,\dots,n$$

考虑到玩家如果在矿山挖矿，消耗的资源数量是基础消耗量的 3 倍，且沙暴日只能原地停留，则会造成玩家的资源不足时，遇到沙暴日无法行走，又没有挖矿，只能原地停留，则该种情况下的如果第 i 天要挖矿就要保证满足这一天的玩家停留在矿山，且玩家停留有可能不挖矿，具体约束如下：

$$r_i \leq f_{in_i} \quad i=1,2,\dots,t$$

(5) 沙暴日在原地停留的约束

根据游戏基本规则（3）和（4），沙漠中所有区域的天气相同，沙暴日必须在原地停留。已知沙暴日的日期集合为 C ，则可以得知玩家沙暴日在地图上的第 j 个区域原地停留 f_{ij} 之和等于 1，即具体约束如下：

$$\sum_{j=1}^n f_{ij} = 1 \quad i \in C$$

(6) 消耗资源数量的约束

根据游戏基本规则（5）和（7），玩家在原地停留一天消耗的资源数量称为基础消耗量，行走一天消耗的资源数量为基础消耗量的 2 倍，挖矿消耗的资源数量为基础消耗量的 3 倍，设第 i 天消耗的资源数量为基础消耗量的 h_i 倍，令第 i 天水资源消耗量为 U_i ，食物消耗量为 V_i ，则表达式如下：

$$\begin{cases} U_i = h_i u_i \\ V_i = h_i v_i \end{cases} \quad i=1,2,\dots,t; j=1,2,\dots,n$$

①根据以上的表达式，当第 i 天消耗的资源数量为 0 时，则表示第 i 天已经

到达终点，即 $h_i = 0$ 时，游戏结束。

②当第 i 天消耗的资源数量为基础消耗量时，即 $h_i = 1$ ，则表示玩家第 i 天在原地停留，且没有挖矿和行走，可表示为：

$$h_i = 1 \Rightarrow \sum_{j=1}^n f_{ij} = 1$$

③当第 i 天消耗的资源数量为基础消耗量的 2 倍时，即 $h_i = 2$ ，则表示玩家第 i 天在行走，可表示为：

$$h_i = 2 \Rightarrow \sum_{j=1}^n f_{ij} = 0 \text{ 且 } \sum_{j=1}^n g_{ij} = 1$$

④当第 i 天消耗的资源数量为基础消耗量的 3 倍时，即 $h_i = 3$ ，则表示玩家第 i 天在挖矿，令矿山的区域编号为 n_1 ，可表示为：

$$h_i = 3 \Rightarrow f_{in_1} = 1$$

结合以上的①②③④，可以得到倍数 h_i 的分段函数表示如下：

$$h_i = \begin{cases} 0, & \sum_{j=1}^n g_{ij} = 0 \\ 1, & \sum_{j=1}^n f_{ij} = 1 \\ 2, & \sum_{j=1}^n f_{ij} = 0 \text{ 且 } \sum_{j=1}^n g_{ij} = 1 \\ 3, & f_{in_1} = 1 \end{cases} \quad i = 1, 2, \dots, t$$

(7) 任意两个区域之间的约束

由于玩家每天可从地图中的某个区域到达与之相邻的另一个区域，已知地图中的任意区域到达另一个区域的邻接矩阵 $D_{n \times n}$ ，则要保证第 i 天所经过第 j 个区域与第 $i+1$ 天所经过第 k 个区域要满足邻接矩阵的相连通关系，则乘积不得大于第 j 个区域到第 k 个区域的权值，具体约束如下：

$$g_{ij} * g_{i+1k} \leq D_{jk} \quad i = 1, 2, \dots, t-1; j = 1, 2, \dots, n; k = 1, 2, \dots, n$$

(8) 在起点购买资源的资金约束

根据游戏基本规则（6），已知玩家第 0 天在起点处用初始资金以基准价格购买水的价格为 x_1 和食物的价格为 x_2 ，设初始资金为 Y_0 ，购买的水和食物的数量（箱）分别为 a_0 和 b_0 ，令第 0 天购买水和食物花费的资金为 Y_1 ，则购买的水和食物的价格不得大于初始资金，具体约束如下：

$$Y_1 = x_1 a_0 + x_2 b_0 \leq Y_0$$

(9) 退回剩余水和食物得到的资金

已知玩家到达终点后可退回剩余的水和食物，每箱退回价格为基准价格的一半。设玩家到达终点后可退回的资金 Y_t ，令玩家达到终点后的日期为 t 时，可以得知第 t 天剩余的水量为 a_t ，食物量为 b_t 。因此，可以得到 a_t 箱水和 b_t 箱食物的退回资金，即退回的资金具体表达式如下：

$$Y_t = \frac{1}{2}(x_1 a_t + x_2 b_t)$$

(10) 挖矿获得的资金

根据游戏基本规则（7），玩家在矿山停留时，可通过挖矿获得资金，挖矿一天获得的资金量称为基础收益，记为 y ，矿山的区域编号为 n_1 ，则挖矿获得的资金 Y_3 等于玩家在挖矿的天数与基础收益的乘积，即表达式如下：

$$Y_3 = \sum_{i=1}^t r_i * y$$

(11) 在村庄购买水和食物的资金约束

根据游戏基本规则（8），玩家经过或在村庄停留时可用剩余的初始资金或挖矿获得的资金随时购买水和食物，每箱价格为基准价格的 2 倍。设第 i 天在村庄购买的水量为 a'_i ，食物量为 b'_i ，村庄在地图上的区域编号为 n_2 ，则第 i 天在村庄购买水和食物的资金不得大于前一天剩余的资金 y_{i-1} ，具体约束如下：

$$2 * g_{in_2}(x_1 a'_i + x_2 b'_i) \leq y_{i-1} \quad i = 1, 2, \dots, t$$

其中，在整个游戏时段内，可以求得玩家经过或在村庄停留时总共购买水和食物的资金，记为 Y_2 ，即表达式如下：

$$Y_2 = 2 * \sum_{i=1}^t g_{in_2}(x_1 a'_i + x_2 b'_i)$$

5.1.4 剩余资金数（剩余水量、剩余食物量）

由于游戏的目标是在规定时间内到达终点，并保留尽可能多的资金。而不同的天气状况、原地停留和挖矿对资源的消耗数量不同，从而导致购买资源数量增加，使到达终点时剩余资金数比较少。为了使到达终点时剩余的资金数最多，确定玩家的最优策略。我们根据下图 2 中影响剩余资金数、剩余水量和剩余食物量的几个主要因素进行具体分析。

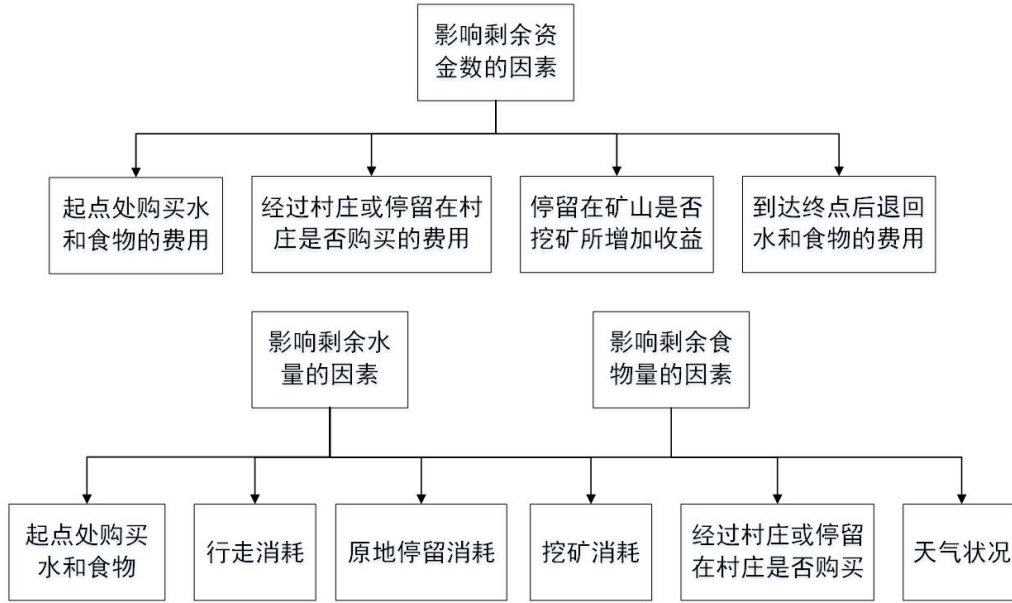


图 2：影响剩余资金数（剩余水量、剩余食物量）主要因素

（1）剩余资金数

为了确定最优策略的具体结果，设第 i 天的剩余资金数为 y_i ，则第 i 天的剩余资金数要等于第 $i-1$ 天的剩余资金数与第 i 天挖矿的收益之和减去第 i 天在村庄购买水和食物花费的资金，具体表达式如下：

$$y_i = y_{i-1} + r_i y - 2 * g_{in_2} (x_1 a'_i + x_2 b'_i) \quad i = 1, 2, \dots, t$$

其中，已知初始资金为 Y_0 ，第0天以基准价格购买水和食物的花费资金为 Y_1 ，当 i 等于1时，则 y_0 等于初始资金减去在第0天购买的水和食物所花费的资金 Y_1 ，则第0天的剩余资金数可得表达式如下：

$$y_0 = Y_0 - Y_1$$

（2）剩余水量

由于已知第 i 天水的基础消耗量为 u_i ，设第 i 天的剩余水量为 a_i ，则第 i 天的剩余水量等于第 $i-1$ 天的剩余水量 a_{i-1} 与第 i 天在村庄购买水量 a'_i 之和减去第 i 天消耗的水资源数量，具体表达式如下：

$$a_i = a_{i-1} - h_i u_i + g_{in_2} a'_i \quad i = 1, 2, \dots, t$$

其中，以上已知第0天在起点处购买的水量为 a_0 ，当 i 等于1时，则 a_0 等于第0天在起点处购买的水的数量（箱）。

（3）剩余食物量

根据以上的每天剩余的求量的求解思路，同理可以得出第 i 天的剩余食物量 b_i 等于第 $i-1$ 天的剩余食物量 b_{i-1} 与第 i 天在村庄购买食物量 a'_i 之和减去第 i 天消耗的食物数量，具体表达式如下：

$$b_i = b_{i-1} - h_i v_i + g_{in_2} b'_i \quad i = 1, 2, \dots, t$$

其中，第 0 天在起点处购买的食物量为 b_0 ，当 i 等于 1 时，则 b_0 表示玩家第 0 天在起点处购买食物的数量（箱）。

5.1.5 模型I的建立与求解

综合上述目标模型和约束条件分析，以玩家在规定时间内到达终点后，剩余的总资金最多为目标，以第 i 天是否到地图上的第 j 个区域 g_{ij} 为变量，建立最优化模型，具体模型如下：

$$\begin{aligned} \text{目标: } \text{Max } z &= Y_0 - Y_1 + \sum_{i=1}^t r_i y - Y_2 + \frac{1}{2}(x_1 a_t + x_2 b_t) \\ s.t. \quad &\begin{cases} t \leq T \\ 0 \leq a_i m_1 + b_i m_2 \leq M & i = 0, 1, \dots, t \\ \sum_{j=1}^n f_{ij} = 1 & i \in C \\ g_{ij} * g_{i+1k} \leq D_{jk} & i = 1, 2, \dots, t-1; j = 1, 2, \dots, n; k = 1, 2, \dots, n \\ 2 * g_{in_2} (x_1 a'_i + x_2 b'_i) \leq y_{i-1} & i = 1, 2, \dots, t \\ Y_1 = x_1 a_0 + x_2 b_0 \leq Y_0 \\ Y_2 = \sum_{i=1}^t 2 * g_{in_2} (x_1 a'_i + x_2 b'_i) \\ y_i = y_{i-1} + r_i y - 2 * g_{in_2} (x_1 a'_i + x_2 b'_i) & i = 1, 2, \dots, t \\ a_i = a_{i-1} - h_i u_i + g_{in_2} a'_i & i = 1, 2, \dots, t \\ b_i = b_{i-1} - h_i v_i + g_{in_2} b'_i & i = 1, 2, \dots, t \\ r_i \leq f_{in_1}; f_{ij} \leq g_{ij} & i = 1, 2, \dots, t; j = 1, 2, \dots, n \\ a_i \in N^*; b_i \in N^* & i = 0, 1, \dots, t-1 \\ g_{ij} \in \{0, 1\}; f_{ij} \in \{0, 1\}; r_i \in \{0, 1\} & i = 1, 2, \dots, t; j = 1, 2, \dots, n \end{cases} \end{aligned}$$

5.1.6 设计最优策略求解算法的具体描述

根据以上建立的最优化模型，利用贪心算法的思想^[3]，设计相对应的求解算法，具体算法步骤如下：

Step1: 初始化游戏信息。根据题目提供的关卡参数初始化各种参数，包括

负重上限 M 、初始资金 Y_0 、截止日期 T 、基础收益 y 、每箱水重量 m_1 、每箱食物重量 m_2 、每箱水价格 x_1 、每箱食物价格 x_2 、根据天气变化的水基础消耗量 u_i 、食物消耗量 v_i 以及所给地图的路径信息等。

Step2: 规划行走路径。首先在所给地图中执行最短路径算法，得到图中起点、村庄、矿山和终点的任意两个地区之间的最短路径。而后人工规划出数条挖矿次数大于 0 的行走路径，以及一条直接抵达终点的路径，并约定不同路径中的挖矿、购买次数。

Step3: 筛选最优路径。给定相同的食物量和水量，结合游戏关卡参数以及天气信息，模拟玩家在规划的路径上行走的过程，并记录行走途中玩家负重、食物量、水量、资金等参数的变化情况。若途中食物量或水量清零，终止玩家行走，切换下一条路径。留出能使玩家抵达终点并且留有较多资金的路径，删除其他路径。

Step4: 遍历初始水量、食物量组合。根据负重上限 M ，初始资金 Y_0 ，计算出玩家单独购买食物或水量时的最大数量，遍历所有数量组合，计算在不同组合下，玩家沿着预定的最优路径抵达终点时的剩余资金数。剩余资金最多的数量组合以及预定的最优路径即为最优决策方案。

Step5: 计算每日剩余信息。依据所得的最优方案以及题目提供的游戏规则，计算出玩家在抵达终点之前，每天的所在区域、剩余资金数、剩余水量、剩余食物量。首先设定好玩家的在起点购买水量与食物量，并计算好剩余资金。然后根据求得的路线方案，计算出玩家每天在相应地区，相应天气做出行走、停留、购买或挖矿行为时的物资消耗量。再计算出方案记录中玩家从一个村庄出发，并再次进入村庄或抵达终点时，中间间隔时间进行一系列行为的资源消耗，将其作为在先前存在出发时需要补充的物资预测值。当计算一天的剩余物资情况时，使用玩家原先的物资数减去当天的物资消耗量，所得结果为当前剩余物资，同时作为玩家下个日期的物资初始值。若所得剩余物资中出现负数，则舍弃当前起点物资组合，更换其他组合。若目前玩家位于村庄，则将玩家初始值加上预测值后再与消耗值相减得到剩余量。若剩余量仍出现负数情况，增加相应村庄的物资补充量。若补充量超出负重上限，则改为增加下个村庄的物资补充量。

Step6: 重复上述步骤，直至玩家在终点的剩余物资计算完毕。

5.1.7 具体结果

综上所述，对 5.1.5 中建立的最优化模型 I，利用 *Matlab* 编程求解，具体程序见附录二，得到本题附件中的“第一关”和“第二关”一般情况下玩家的最优策略，和该策略下玩家在规定时间内到达终点后剩余的总资金最多。

(1) 第一关的具体结果

结合附件中的“第一关”的参数设定和地图中的区域与区域之间邻接矩阵，可以得知两种不同玩家行走方案的策略，①第一种方案是玩家刚好在截止日到达终点，到达终点后剩余的总资金最多为 10230 元，该过程中挖矿 9 天；②第二种方案是玩家在 23 天时到达终点，到达终点后剩余的总资金最多为 10430 元，该过程中挖矿 7 天，则可以得知第二种方案的行走策略是最优策略，而得到这种最优结果的策略也会出现多条最优行走路线，即一般情况下玩家的最优策略的两种最优路线规划图如下：

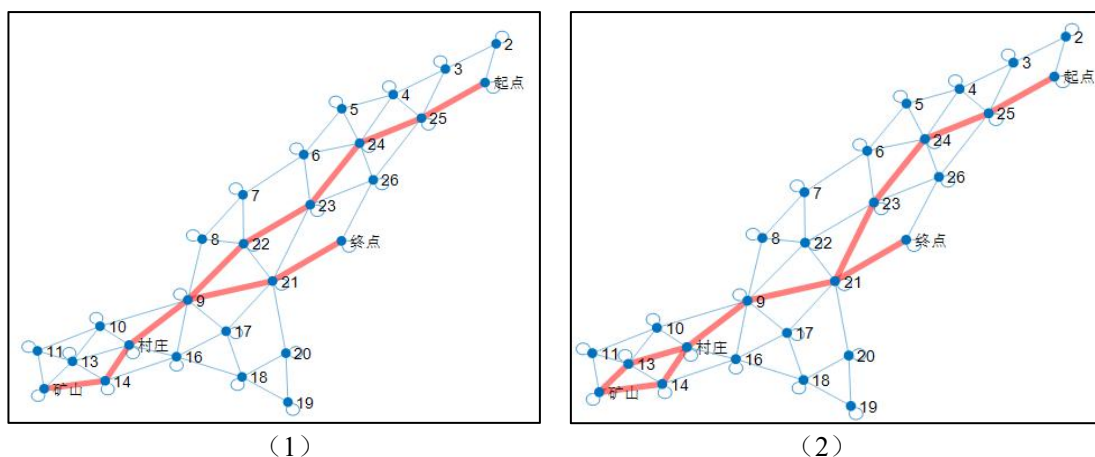


图 3：玩家的两条最优路线规划图

根据以上的两条最优行走路线，可以得到两条线路在规定时间内到达终点后剩余的总资金最多为 10430 元，则玩家第 0 天在起点处购买的水量必须为 180 箱，食物量为 330 箱，其中到达终点后剩余的水量和食物量均为 0 箱，该策略中玩家需要在村庄购买两次水；即第一关玩家的最优策略所对应具体结果详见数据见支撑材料 Result1.xlsx 的第一关，其部分结果见下表 1 所示：

表 1：第一关的部分结果（以第（1）条路线为例）

日期	所在区域	剩余资金数	剩余水量	剩余食物量
0	1	5800	180	330
1	25	5800	164	318
⋮	⋮	⋮	⋮	⋮
8	15	4170	245	232
⋮	⋮	⋮	⋮	⋮
23	27	10430	0	0

(2) 第二关的具体结果

结合附件中的“第二关”的参数设定和地图中的区域与区域之间邻接矩阵，可以得到一般情况下玩家的最优策略有多条规划路线，其中两种最优路线规划图如

下图 4 所示：

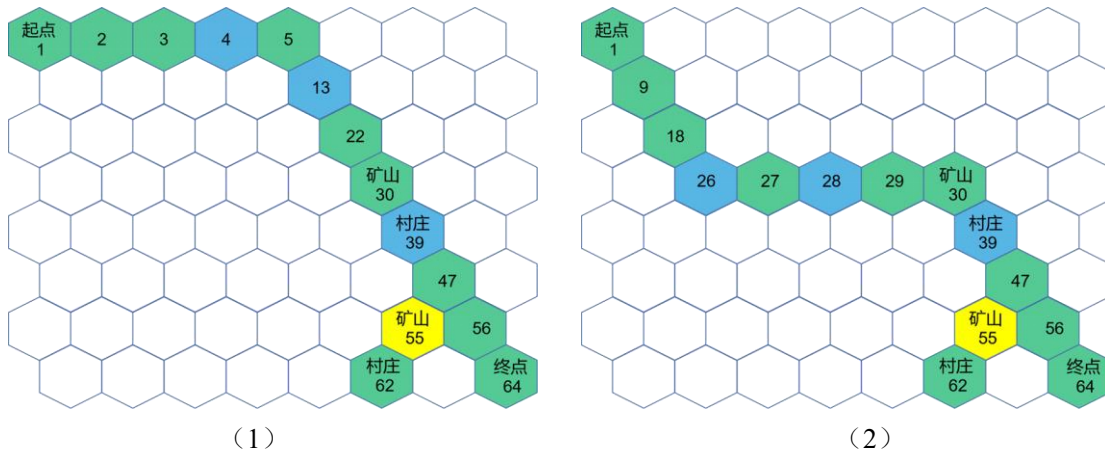


图 4：玩家的两条最优路线规划图

根据以上的两条最优行走路线，可以发现第二关玩家根据这两条行走路线的最优策略所得到对应具体结果相同，且得到玩家第 0 天在起点处购买的水量为 130 箱，食物量为 405 箱时，在规定时间内到达终点后剩余的总资金最多为 12730 元，其中到达终点后剩余的水量和食物量均为 0 箱，且玩家需要在村庄购买两次水，一次食物，详见数据见支撑材料 Result1.xlsx 的第二关，其部分结果见下表 2 所示：

表 2：第二关的部分结果（以第（1）条路线为例）

日期	所在区域	剩余资金数	剩余水量	剩余食物量
0	1	5300	130	405
1	2	5300	114	393
⋮	⋮	⋮	⋮	⋮
18	55	8190	43	133
19	62	4730	201	207
⋮	⋮	⋮	⋮	⋮
30	64	12730	0	0

5.2 问题 2：仅知道当天天气状况的最佳策略

在本问中，需要在问题 1 的模型 I 基础上，考虑单人游戏中玩家仅知道当天的天气状况，从而决定当天的行动方案，添加约束条件，以在规定时间内到达终点后所保留的资金最多为目标，建立最优化模型，设计算法，分别求解附件中的“第三关”和“第四关”的一般情况下玩家的最优策略。

5.2.1 地图中区域到达另一个区域的邻接矩阵

结合附件中第三和第四关所给的地图，则可以得到地图中的任意区域到达另一个区域的邻接矩阵 D ，具体数据分别见支撑材料附件 1 的第三关和第四关。由于任意两个区域可以来回多次经过，则可以确定所给地图是属于无向图，而无

向图的邻接矩阵是对称矩阵，可以得到第三关和第四关的无向连通图如下图 5 所示：

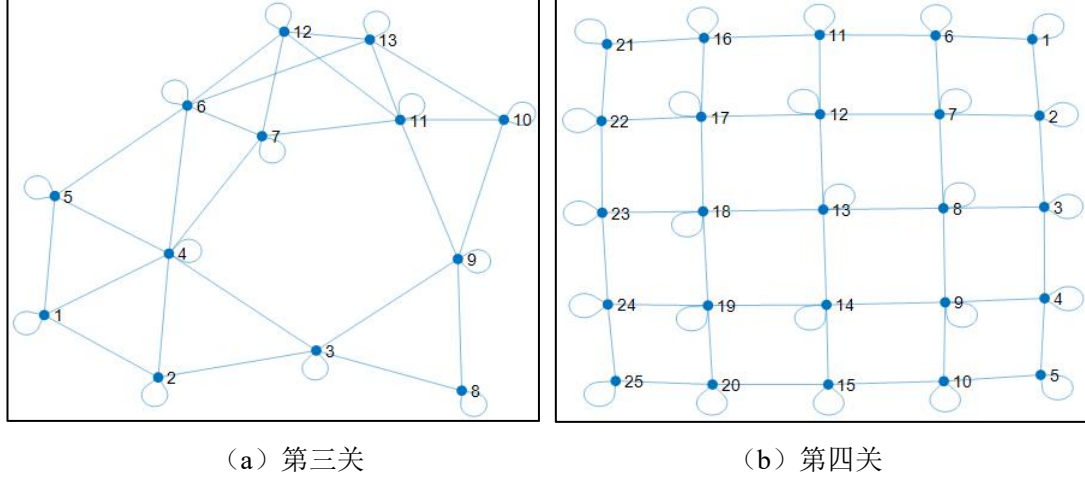


图 5：无向连通图

5.2.2 添加约束条件

为了根据当天的天气状况决定当天的行动方案，保证玩家在规定时间内可以到达终点，需要添加如下约束如下：

(1) 每天剩余资源的约束

且该过程中，不管遇到何种天气状况都不会出现未到达终点时，水或食物已耗尽的情况，则需要约束每天剩余的水量、食物量和资金数要尽可能多，不会出现供不应求的情况，即具体约束如下：

$$w_i = \max(a_i, b_i, y_i) \quad i = 0, 1, \dots, t$$

(2) 出现沙暴状况的概率

由于第三关中玩家仅知道当天的天气状况，但已知的 10 天内不会出现沙暴天气，而其截止日期就是第 10 天，则表示第三关中出现沙暴天气的概率为 $P = 0$ ；而第四关中玩家已知 30 天内较少出现沙暴天气，则假设已知的 30 天内出现沙暴天气的概率为 $P = 1 \sim 10\%$ ，采用随机模拟确定当天是否出现沙暴天气，令出现沙暴天气的集合为 C ，即

$$P(i \in C) = \kappa \quad i = 1, 2, \dots, t$$

(3) 挖矿天数和经过矿点的约束

已知玩家第 i 天是否挖矿为 r_i ，第 i 天是否在第 j 个区域为 g_{ij} ，其中矿山所在的区域为 n_1 ；由于到达矿山当天不能挖矿，所以在整个游戏时间内，挖矿的天数不能大于玩家在矿山的天数与 1 的差值，具体约束如下：

$$\sum_{i=1}^t r_i \leq \sum_{i=1}^t g_{in_1} - 1$$

5.2.3 模型II的建立与求解

结合以上添加约束条件分析，调整问题 1 玩家的最优策略模型，以玩家在规定时间内到达终点后，剩余的总资金最多为目标，以玩家第 i 天是否在第 j 个区域 g_{ij} 为变量，建立最优化模型II，具体模型如下：

$$\begin{aligned} \text{目标: } \text{Max } z &= Y_0 - Y_1 + \sum_{i=1}^t r_i y - Y_2 + \frac{1}{2}(x_1 a_t + x_2 b_t) \\ s.t. \left\{ \begin{array}{ll} w_i = \max(a_i, b_i, y_i) & i = 0, 1, \dots, t \\ P(i \in C) = \kappa & i = 1, 2, \dots, t \\ t \leq T \\ 0 \leq a_i m_1 + b_i m_2 \leq M & i = 0, 1, \dots, t \\ g_{ij} * g_{i+1k} \leq D_{jk} & i = 1, 2, \dots, t-1; j = 1, 2, \dots, n; k = 1, 2, \dots, n \\ 2 * g_{in_2} (x_1 a'_i + x_2 b'_i) \leq y_{i-1} & i = 1, 2, \dots, t \\ \sum_{i=1}^t r_i \leq \sum_{i=1}^t g_{in_1} - 1 \\ Y_1 = x_1 a_0 + x_2 b_0 \leq Y_0 \\ Y_2 = 2 * \sum_{i=1}^t g_{in_2} (x_1 a'_i + x_2 b'_i) \\ r_i \leq g_{in_1} & i = 1, 2, \dots, t \\ a_i \in N^*; b_i \in N^* & i = 0, 1, \dots, t-1 \\ g_{ij} \in \{0, 1\}; r_i \in \{0, 1\} & i = 1, 2, \dots, t; j = 1, 2, \dots, n \end{array} \right. \end{aligned}$$

5.2.4 设计最佳策略求解算法的具体描述

根据以上建立的最优化模型II，结合蒙特卡洛随机模拟^[5]当天的天气状况，结合贪心算法的思想，具体算法步骤如下：

Step1: 初始化游戏信息。根据题目所给数据设置好相关游戏参数。

Step2: 生成天气信息。根据游戏截止日期，采用蒙特卡洛法随机生成相应天数的天气信息。

Step3: 寻找最佳路径。根据所给题目所给地图，首先计算出地图里任意两个地区的最短路径。而后预先让玩家持有相同的水量和食物量，根据目前遭遇的

天气情况，做出下一步选择。若玩家遭遇晴朗或高温天气，则根据剩余游戏时间情况，优先赶往矿山获取更多资金，若剩余物资较少，则优先前往村庄购买物资。若玩家遇到沙暴天气，优先选择原地停留，如果玩家在矿山地区，根据剩余物资情况优先选择挖矿。若剩余时间已经不足以做其他活动，则前往终点。最后根据能到达终点的路径决策信息，记录下不同路径决策的最终剩余资金情况，保留剩余资金最多的路径决策。

Step4: 遍历水量、食物量组合。根据初始参数求出玩家在起点可获得的最大食物量与水量。遍历两者不同的数量组合，将其设定为玩家的初始物资携带量，令玩家沿着预定的最优路径开始行走。若玩家可以抵达终点，则计算玩家此时剩余的资金数，剩余资金数最多的数量组合与预定的路径决策即为最优决策方案。

Step5: 计算每日剩余信息。依据所得的最优方案以及题目提供的游戏规则，计算出玩家在抵达终点之前，每天的所在区域、剩余资金数、剩余水量、剩余食物量（详细计算规则如问题一算法 Step5 所述）。

5.2.5 具体结果

综上所述，根据 5.2.4 的具体算法求解步骤，利用 *Matlab* 编程实现，具体程序见附录二，通过规划三种不同情况下的天气状况决定玩家的行动策略，求解本题附件中的“第三关”和“第四关”的一般玩家的最佳策略，得到玩家在规定时间内到达终点后剩余的总资金最多。

(1) 第三关的具体结果

结合附件中的“第三关”的参数设定和地图中的区域与区域之间邻接矩阵，已知 10 天内不会出现沙暴天气，需要分三种不同的天气状况规则方案进行具体讨论研究玩家行走方案的策略。由于三种情方案的玩家的行走路线的相同，则可以得到玩家的最优策略中两种最优路线规划图如下图 6 所示：

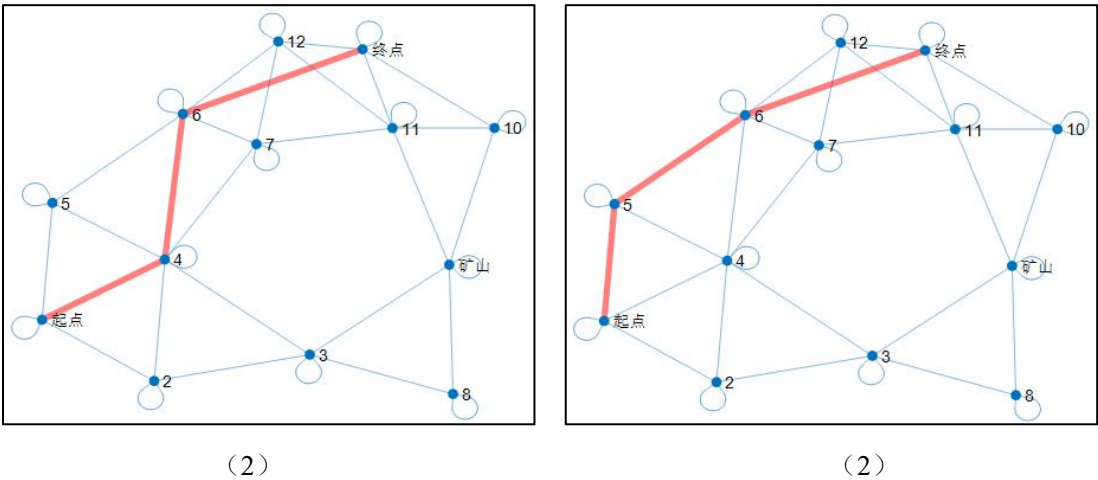


图 6：玩家的两条最优路线规划图

①天气状况全部高温的方案

第一种情况：假设在整个游戏中每天的天气状况均为高温天气时，由于通过附件的参数设定已知高温天气的水和食物的基础消耗量均比晴朗天气大，则约束购买的水和食物刚够玩家在最短路线中的消耗即可，求得玩家在规定时间内到达终点后，剩余的总资金最多为 9190 元，该种情况下，不会出现到达终点退回水和食物的情况，且没有出现到矿山挖矿的情况，即全部高温的天气状况的玩家最佳策略的对应结果见下表 3 所示：

表 3：全部高温情况的结果（以第（1）条路线为例）

日期	天气状况	所在区域	剩余资金数	剩余水量	剩余食物量
0		1	9190	54	54
1	高温	4	9190	36	36
2	高温	6	9190	18	18
3	高温	13	9190	0	0

为了验证以上的路线是否是玩家的最佳策略，研究玩家准备足够的水和食物到达矿山挖矿后，在规则时间内，按照最短路线求解玩家到达终点后的剩余总资金最多为 7625 元，即可以得知玩家过去矿山挖矿后，在规定时间内到达终点后剩余的总资金比玩家直接到达终点后的总资金少，因此后面验研究的两种不同天气情况不需要考虑去矿山挖矿的情况。

②天气状况全部晴朗的方案

第二种情况：假设在整个游戏中每天的天气状况均为晴朗天气，求得玩家在规定时间内到达终点后，剩余的总资金最多为 9430 元，该种情况下，为了保证游戏不会出现失败，玩家在起点处购买资源时，得预留一部分水和食物保证游戏的顺利通关，则到达终点后会退回一部分水和食物，即全部晴朗的天气状况的玩家最佳策略的对应结果见下表 3.1 所示：

表 3.1：全部晴朗情况的结果（以第（1）条路线为例）

日期	天气状况	所在区域	剩余资金数	剩余水量	剩余食物量
0		1	9190	54	54
1	高温	4	9190	48	46
2	高温	6	9190	42	38
3	高温	13	9430	0	0

③一般情况的天气状况的方案

第三种情况：假设在整个游戏中每天的天气状况未确定，按照随机模拟天气的情况，由于玩家三天就可以直接到达终点，则只考虑三天天气，当出现有两天的

晴朗，一天高温时，玩家在规定时间内到达终点后，剩余的总资金最多为 9350 元；出现两天高温，一天晴朗时，玩家剩余的总资金最多为 9270 元，取资金相对比较大的为例，得到一般情况的天气状况的玩家最佳策略的对应结果见下表 3.2 所示：

表 3.2：一般天气情况的结果（以第（1）条路线为例）

日期	天气状况	所在区域	剩余资金数	剩余水量	剩余食物量
0		1	9190	54	54
1	晴朗	4	9190	48	46
2	高温	6	9190	30	28
3	晴朗	13	9350	0	0

（2）第四关的具体结果

结合附件中的“第四关”的参数设定和地图中的区域与区域之间邻接矩阵，已知 30 天内较少出现沙暴天气，假设出现沙暴的概率为 1~10%，采用蒙特卡洛随机算计模拟出现沙暴的日期，研究 30 天内大部分是高温、晴朗和一般情况的 3 种不同的天气状况，对此种情况下一般情况玩家的最佳策略进行讨论。

①天气状况为极端高温的方案

第一种情况：假设在整个游戏中出现高温天气的占据大部分时，由于已知高温天气的水和食物的基础消耗量比晴朗天气大，且水和食物的基础消耗量相同，则直接定第 0 天平均购买的水和食物的数量即可，求得玩家将在第 20 天到达终点后保留的资金比较多，剩余的总资金最多为 8787.5 元，即极端高温的天气状况的玩家最佳策略的对应结果详见支撑材料 Result_2 的表 1：

②天气状况为极端晴朗的方案

第二种情况：假设在整个游戏中出现晴朗天气的占据大部分时，考虑预计挖矿准备走人时至少会出现一天沙暴，可能出现 2 天高温才到村庄，3 天高温才能到达终点，因此得提前预留一部分水和食物保证游戏的顺利通关，求得玩家将在第 29 天到达终点后保留的资金比较多，剩余的总资金最多为 20012.5 元，即极端晴朗的天气状况的玩家最佳策略的对应结果详见支撑材料 Result_2 的表 2：

③一般情况的天气状况的方案

第三种情况：假设在整个游戏中每天的天气状况按照比例均匀随机模拟决定天气是高温还是晴朗的情况，预计玩家从矿山到村庄，到终点只可能出现一次沙暴的情况，由于玩家从矿山到村庄最短路径需要 2 天，到终点最短路径需要 3 天，可能不会出现晴朗的情况，得提前预留足够的水和食物保证游戏可以顺利通关，求得玩家在规定时间内到达终点后，剩余的总资金最多为 12622.5 元，相对应的结果见下表 3.3 所示：

表 3.3：一般天气情况的结果（以第（1）条路线为例）

日期	天气状况	所在区域	剩余资金数	剩余水量	剩余食物量
0		1	6400	240	240
1	高温	2	6400	222	222
2	高温	3	6400	204	204
⋮	⋮	⋮	⋮	⋮	⋮
12	高温	14	6890	235	230
⋮	⋮	⋮	⋮	⋮	⋮
26	高温	20	12370	49	37
27	晴朗	25	12622.5	0	0

根据以上得到一般情况的天气状况的玩家最佳策略的对应具体结果详见支撑材料 Result_2 的表 3，由于玩家的最优策略中可能会出现多条最优路线，其中两种最优路线规划图如下图 7 所示：

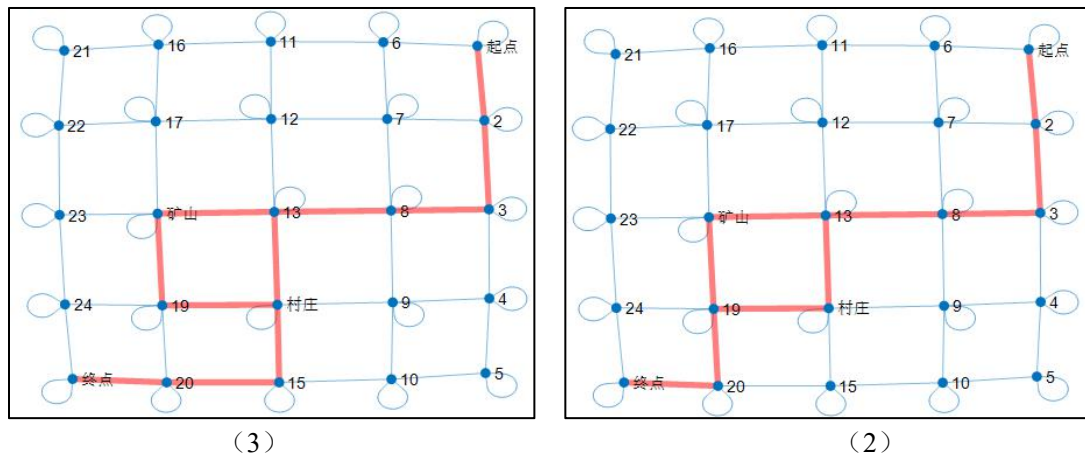


图 7：玩家的两条最优路线规划图

以上两种最佳路线规划中的策略下第 0 天玩家购买水和食物均为 240 箱，第 1 天开始直接按最短路径到达矿山挖矿，在第 13 天和 24 天会到达村庄购买水和食物，在第 17 天会出现沙暴日，玩家将在第 27 天到达终点，游戏结束。

5.3 问题 3：一般情况下多名玩家应采取的策略

在本问中，需要研究两个小问，已知多名玩家拥有相同的初始资金，且同时从起点出发。需对多名玩家在行走的过程中可能遇到的情况下，每位玩家行走一天消耗资源数量是单人游戏时的 k 倍数、挖矿获得资金的 $1/k$ ，在村庄购买资源时，是基准价格的 4 倍；根据该规则调整前两问建立的最优化模型 I 和 II，考虑第 1 小问每天天气状况事先已知的“第五关”和第 2 小问所有玩家仅知道当天的天气状况的“第六关”的最佳采用策略，调用调整后的模型，分别采用多种情况讨论玩家的最佳策略，

5.3.1 第 1 小问：每天天气状况事先已知的情况

在第 1 小问中，考虑的是整个游戏时段内每天天气状况事先全部已知的多人游戏，以规定时间内所有玩家到达终点后，剩余的总资金最多为目标，调整问题 1 的最优化模型 I，设计一般情况下玩家采取策略，对附件中的“第五关”得到的相应结果进行讨论。

5.3.1.1 更改模型 I 的目标

由于出现多名玩家后，消耗的资源数量、挖矿获得的资金和村庄购买水和食物的费用都会发生改变，故无法直接保证单人保留的资金最多即可，则以本问中所有玩家在规定时间内到达终点后，剩余的总资金最多为目标，作相应的更改，更改后的具体目标如下：

$$\text{Max } z = \sum_{k=1}^p \left(Y_{k0} - Y_{k1} + Y_{k3} - Y_{k2} + \frac{1}{2}(x_1 a_{kt} + x_2 b_{kt}) \right)$$

注：题意指的现有 n 名玩家的符号 n 改为符号 p ，以下表达式均是。

5.3.1.2 添加和修改约束条件

为了使游戏增加多名玩家时，上述问题 1 建立的模型 I 同样适用，需对目标添加和修改如下条件：

(1) 每天每个区域的玩家人数约束

结合题意，为了确定每天每个区域的玩家人数，设第 i 天经过第 j 个区域的玩家人数为 s_{ij} ，根据第 k 玩家与第 l 个玩家是否在同一天同个区域 g_{ijk} 为判断条件，可以得到每天每个区域的玩家人数的约束条件如下：

$$s_{ij} = \begin{cases} s_{ij} + 1 & g_{ijk} = g_{ijl} \quad k = 1, 2, \dots, p; l = k + 1, k + 2, \dots, p \\ s_{ij} + 0 & g_{ijk} \neq g_{ijl} \quad i = 1, 2, \dots, t; j = 1, 2, \dots, n \end{cases}$$

(2) 修改消耗资源数量的倍数约束

根据问题 1 的消耗资源数量的约束条件中得到倍数 h_i 的分段函数，由于第 i 天任意 $k(2 \leq k \leq p)$ 名玩家均从区域 A 行走到区域 $B (B \neq A)$ 时，他们中任意一位消耗的资源数量均为基础消耗量的 $2k$ 倍，若已知第 i 天在第 j 个区域的玩家人数为 s_{ij} ，则可以修改得到第 k 个玩家在第 i 天所消耗的资源数量的倍数 h_i 的关系，即表达式如下：

$$h_{ik} = \begin{cases} 0, & \sum_{j=1}^n g_{ijk} = 0 \\ 1, & \sum_{j=1}^n f_{ijk} = 1 \\ 2 * s_{ij}, & \sum_{j=1}^n f_{ijk} = 0 \text{ 且 } \sum_{j=1}^n g_{ijk} = 1 \\ 3, & f_{in_i k} = 1 \end{cases} \quad k=1,2,\dots,p; i=1,2,\dots,t$$

(3) 多名玩家相遇的购买价格倍数约束

由于某天其中的任意 $k(2 \leq k \leq p)$ 名玩家在同一村庄购买资源, 每箱价格均为基准价格的 4 倍; 但只有一名玩家在村庄购买资源, 其价格和单人游戏的情况一样为 2 倍; 若已知第 i 天经过第 j 个区域的玩家人数为 s_{ij} , 根据第 i 天第 j 个区域的玩家人数是否大于 1 进行判断, 可以所有玩家在第 i 天第 j 个区域购买资源的价格是基准价格的 q_{ij} 倍数, 即约束条件如下:

$$q_{ij} = \begin{cases} 2 & s_{ij} \leq 1 \\ 4 & s_{ij} > 1 \end{cases} \quad i=1,2,\dots,t; j=1,2,\dots,n$$

(4) 修改挖矿获得的资金

根据游戏基本规则, 由于某天其中的任意 $k(2 \leq k \leq p)$ 名玩家在同一矿山挖矿时, 每名玩家一天可通过挖矿获得的资金是基础收益的 $1/k$, 已知第 i 天经过第 j 个区域的玩家人数为 s_{ij} , 矿山的区域编号记为 n_1 , 则对玩家挖矿获得的资金 Y_{k3} 等于玩家在挖矿的天数与挖矿一天获得资金的乘积作相应的更改, 更改后的表达式如下:

$$Y_{k3} = \frac{1}{s_{in_1}} \sum_{i=1}^t r_{ik} * y \quad k=1,2,\dots,p$$

(5) 调整在村庄购买水和食物的资金约束

由于单人玩家经过或在村庄停留时可用剩余的初始资金或挖矿获得的资金随时购买水和食物, 每箱价格为基准价格的 2 倍, 而多人同时购买时, 每箱价格为基准价格的 4 倍, 记村庄的区域编号为 n_2 , 在对玩家第 i 天在村庄购买水和食物的资金不得大于前一天剩余的资金 y_{i-1} 作相应的调整, 具体约束如下:

$$q_{in_2} * g_{in_2 k} (x_1 a'_{ik} + x_2 b'_{ik}) \leq y_{i-1k} \quad i=1,2,\dots,t; k=1,2,\dots,p$$

其中, 在整个游戏时段内, 可以求得每位玩家经过或在村庄停留时总共购买水和食物的资金, 记为 Y_{k2} , 即表达式如下:

$$Y_{k2} = \sum_{i=1}^t q_{in_2} * g_{in_2k} (x_1 a'_{ik} + x_2 b'_{ik}) \quad k=1,2,\dots,p$$

5.3.1.4 调整最优化模型I

结合以上添加和修改约束条件，调整问题 1 玩家的最优化模型I，具体模型如下：

$$\begin{aligned} \text{目标: } \text{Max } z &= \sum_{k=1}^p \left(Y_{0k} - Y_{1k} + Y_{3k} - Y_{2k} + \frac{1}{2} (x_1 a_{tk} + x_2 b_{tk}) \right) \\ s.t. \quad & \begin{cases} s_{ij} = \begin{cases} s_{ij} + 1, & g_{ijk} = g_{ijl} \\ s_{ij} + 0, & g_{ijk} \neq g_{ijl} \end{cases} & k=1,2,\dots,p; l=i+1, i+2, \dots, p; \\ & i=1,2,\dots,t; j=1,2,\dots,n \\ g_{ijk} * g_{i+1lk} \leq D_{jl} & k=1,2,\dots,p; i=1,2,\dots,t-1; j=1,2,\dots,n; l=1,2,\dots,n \\ t < T \\ 0 \leq a_{ik} m_1 + b_{ik} m_2 \leq M & i=0,1,\dots,t \\ q_{ij} * g_{in_2k} (a'_{ik} x_1 + b'_{ik} x_2) \leq y_{i-1k} & k=1,2,\dots,p; i=1,2,\dots,t \\ Y_{k1} = a_{k0} x_1 + b_{k0} x_2 \leq Y_0 & k=1,2,\dots,p \\ Y_{k2} = \sum_{i=1}^t q_{in_2} * g_{in_2k} (a'_{ik} x_1 + b'_{ik} x_2) & k=1,2,\dots,p \\ y_{ik} = y_{i-1k} + \frac{1}{s_{in_1}} r_{ik} y - q_{ij} * g_{in_2k} (a'_{ik} x_1 + b'_{ik} x_2) & k=1,2,\dots,p; i=1,2,\dots,t \\ a_{ik} = a_{i-1k} - h_{ik} u_i + g_{in_2k} a'_i & k=1,2,\dots,p; i=1,2,\dots,t \\ b_{ik} = b_{i-1k} - h_{ik} v_i + g_{in_2k} b'_i & k=1,2,\dots,p; i=1,2,\dots,t \\ r_{ik} \leq f_{ijk}; f_{ijk} \leq g_{ijk} & k=1,2,\dots,p; i=1,2,\dots,t; j=1,2,\dots,n \\ a_{ik} \in N^*; b_{ik} \in N^* & k=1,2,\dots,p; i=1,2,\dots,t-1 \\ g_{ijk} \in \{0,1\}; r_{ik} \in \{0,1\} & k=1,2,\dots,p; i=1,2,\dots,t; j=1,2,\dots,n \end{cases} \end{aligned}$$

5.3.1.5 设计最佳采取策略求解算法的具体描述

根据以上建立的最优化模型II，已知玩家人数为 2 名，利用贪心算法的思想，设计相对应的求解算法，具体算法步骤如下：

Step1: 初始化游戏信息。根据题目所给数据设置好相关游戏参数。

Step2: 筛选最优路径。首先根据地图信息计算出任意两个地区之间的最短

路径。再为两名玩家设定相同的食物量与水量，令两名玩家各自根据另一名玩家的行动方案，当前天气情况，剩余物资情况，优先选择不同的行进路线。当两名玩家均位于村庄时，其中一名玩家优先选择错开够买物资时间，减少资金消耗。当两名玩家均位于矿山时，优先选择挖矿。在时间不充裕的情况下，两名玩家优先选择前往终点。每名玩家对天气的处理如问题二算法的 Step3 所述。若两名玩家可以抵达终点，根据行走过程，计算此时玩家的剩余资金数和，同时保留资金数和最大的行走路径作为最优路径。

Step3: 遍历水量、食物量组合。根据初始参数求出玩家在起点可获得的最大食物量与水量。遍历两者不同的数量组合，将其设定为玩家的初始物资携带量，令玩家沿着预定的最优路径开始行走。若玩家可以抵达终点，则计算玩家此时剩余的资金数，剩余资金数最多的数量组合与预定的路径决策即为最优决策方案。

Step4: 计算每日剩余信息。依据所得的最优方案以及题目提供的游戏规则，计算出玩家在抵达终点之前，每天的所在区域、剩余资金数、剩余水量、剩余食物量详细计算规则如问题一算法 Step5 所述）。

5.3.1.6 具体结果

综上所述，根据 5.3.1.5 的具体算法求解步骤，利用 *Matlab* 编程实现，已知第五关和第三关的邻接矩阵相同，玩家人数为 2 名，每天天气状况事先全部已知，由于玩家的行动方案需在第 0 天确定，由于玩家如果相遇行走消耗的资源会增多、挖矿获得资金会减少，购买资源费用会增加，因此本问对玩家 3 种行动方案进行具体讨论分析，确定玩家应采取的策略。

①行动方案一

当每个玩家均按照自己的最优路线规划减少相遇情况的行动方案时，求得该策略下两个玩家在规定时间内到达终点后剩余的总资金最多为 18800 元，其对应一般情况下玩家应采取的策略具体结果详见支撑材料 Result_3_1.xlsx。

由于该策略中没有考虑到矿山挖矿的情况，为了验证以上的采取的策略是否是多人玩家中的最佳策略，还需要研究玩家去挖矿的情况，即行动方案二。

①行动方案二

考虑到每个玩家相遇时，消耗的资源比较大，假设玩家 1 按照自己的最优路线规划行动方案时，玩家 2 按照最短路径去挖矿获得资金，按照两个玩家分开行动的情况不会出现相遇，且实现了整个游戏中有玩家到矿山挖矿的情况，利用 *Matlab* 编程可以求得该策略下两个玩家在规定时间内到达终点后剩余的总资金最多为 18175 元，这种行动方案相对应的一般情况下玩家应采取的策略具体结果详见支撑材料 Result_3_1.xlsx。

由于该策略下的总资金比两个玩家均没有挖矿时还少，则需要研究两个玩家

均不挖矿，且尽量不要出现相遇情况，由于如果不挖矿玩家按照最短路径到达终点只需要 3 天，则需要按照玩家不相遇，且目标最大确定行动方案三。

①行动方案三

当玩家 1 按照自己的最优路线规划行动方案后，玩家 2 需要选择除了此路线外的最佳行动方案，使两个玩家的剩余的资金之和最多，其行动方案相对应的一般情况下玩家应采取的策略具体结果见表 4.2 所示：

表 4.2：行动方案三的结果

玩家 1					玩家 2				
日期	所在区域	剩余资金数	剩余水量	剩余食物量	日期	所在区域	剩余资金数	剩余水量	剩余食物量
0	1	9510	30	34	1	9400	36	42	1
1	5	9510	24	26	4	9400	30	34	4
2	6	9510	6	8	7	9400	12	16	7
3	13	9510	0	0	12	9400	6	8	12
游戏已结束					13	9400	0	0	13

根据以上的采取策略可以求得该策略下两个玩家在规定时间内到达终点后剩余的总资金最多为 18910 元，由于该策略下的总资金比其他两个方案的总资金都多，则我们可以得到结论：在“第五关”中，两个玩家应采取以上的行动策略得到的总资金是最多的，是一般情况下玩家的最优策略。

5.3.2 第 2 小问：仅知道当天天气状况的情况

在第 2 小问中，考虑的是玩家仅知道当天的天气状况的多人游戏，每名玩家在当天行动结束后均知道其余玩家当天的行动方案和剩余的资源数量，随后确定各自第二天的行动方案，按照以上第 1 小问的模型调整规则对问题 2 的最优化模型II进行调整，通过规划所有玩家行动方案，确定玩家采用的策略，对附件中的“第六关”得到的相应结果进行具体讨论。

考虑到所有玩家仅知道当天的天气状况，在 5.3.1.4 的模型基础上，增加玩家不知道所有天气的约束条件，参考问题 2 的模型II，综合调整，可以得到多名玩家仅知道当天的天气状况的调整模型，考虑到该模型主要是第 1 小问和问题 2 最优化模型的结合，可忽略不写；

5.3.2.1 设计玩家采取策略的求解算法

假设在整个游戏中每天的天气状况按照比例均匀随机模拟决定天气是高温还是晴朗的情况，已知已知玩家人数为 3 名，结合蒙特卡洛随机模拟当天的天气状况，设计相对应的求解算法，具体算法步骤如下：

Step1：初始化游戏信息。根据题目所给数据设置好相关游戏参数。

Step2: 生成天气信息。根据游戏截止日期，采用蒙特卡洛法随机生成相应天数的天气信息。

Step3: 生成决策信息。预先设定三名玩家具有相同的食物量和水量，使每个玩家根据其他玩家的位置，行动方案，资源剩余数，以及当前的天气情况做出下一步决策。若玩家与其他玩家同时位于村庄，则在资源剩余允许的情况下，错开自身与其他玩家的购买物资时间。若玩家与其他玩家同时位于矿山，优先选择挖矿，其次选择回到村庄购买物资。若玩家是在正常地区行走，则优先选择避开和其他玩家一同行走。每名玩家对天气的处理如问题二算法的 Step3 描述。在剩余时间不足以进行除行走以为的其他活动时，每名玩家都需优先选择前往终点。重复这里的 Step3 步骤，直至每名玩家都抵达终点。最后记录下每名玩家此时的剩余资金情况，保留剩余资金最多的一组决策路径。

Step4: 遍历水量、食物量组合。根据初始参数求出玩家在起点可获得的最大食物量与水量。遍历两者不同的数量组合，将其设定为玩家的初始物资携带量，令三名玩家根据保留的决策路径开始行走，当三名玩家均抵达终点后，计算出各自的剩余资金数，剩余资金数总和最多的数量组合与预定的路径决策即为最优决策方案。

Step5: 计算每日剩余信息。依据所得的最优方案以及题目提供的游戏规则，计算出玩家在抵达终点之前，每天的所在区域、剩余资金数、剩余水量、剩余食物量详细计算规则如问题一算法 Step5 所述）。

5.3.2.2 具体结果

综上所述，已知第六关和第四关的邻接矩阵相同，玩家人数为 3 名，根据 5.3.3 的具体算法求解步骤，利用 *Matlab* 编程实现，随机模拟第二天的天气状况，需要提前预计玩家从矿山到村庄，到终点可能出现沙暴的情况，提前预留足够的水和食物保证游戏可以顺利通关的前提下，使所有玩家剩余资金之和最多，其第 1 个玩家和第 2 个玩家应采取的策略部分结果见表 5 所示：

表 5：玩家 1 和玩家 2 对应的部分结果

玩家 1					玩家 2				
日期	所在区域	剩余资金数	剩余水量	剩余食物量	日期	所在区域	剩余资金数	剩余水量	剩余食物量
0	1	6400	240	240	0	1	6400	240	240
1	6	6400	222	222	1	2	6400	204	204
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
22	20	8733.34	24	18	19	20	6426.67	37	28
23	25	8748.34	0	0	20	25	6524.17	0	0

根据以上一般情况下玩家应采取的策略，求得第 1 名玩家将在第 23 天到达终点后剩余的资金最多约为 8748.33 元，第 2 名玩家将在第 20 天到达终点后剩余的资金最多约为 6524.17 元，第 3 名玩家将在第 18 天到达终点后剩余的资金最多约 7274.17 元，即所有玩家剩余的总资金最多为 22546.67 元，其对应的一般情况下所有玩家应采取的策略具体结果详见支撑材料 Result_3_2.xlsx。

5.3.3 第 0 天购买水量与食物量的灵敏度分析

在不同游戏关卡中，玩家初始购买的水量与食物量对最终的剩余资金数的大小具有一定的影响，考虑变动第六关玩家 1 的初始食物量与水量，观察两者对最终三位玩家的剩余资金总和的影响。其中资金数在食物量与水量于 150~240 箱范围内的变化图如下：

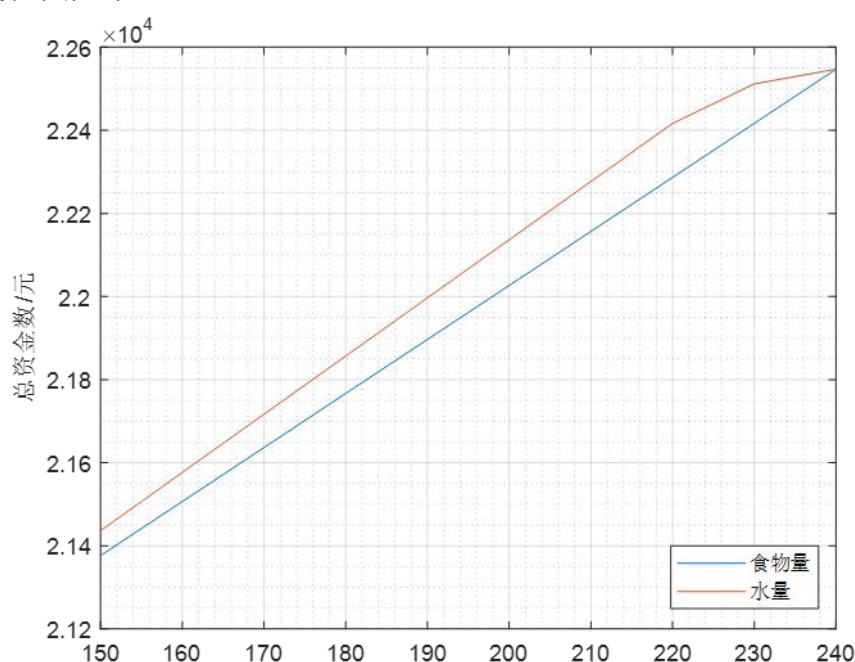


图 8: 灵敏度分析图

根据以上第 0 天购买水量与食物量的灵敏度分析图，可以得到随着初始食物量的增加，最终得出三名玩家的资金总和也在增大，则初始食物量的数值对结果总资金的数值影响大，灵敏度高，初始水量对总资金亦具有相同影响。即初始食物量与水量的灵敏度均较大。

六、模型评价与推广

6.1 模型优点

- 1) 本模型考虑游戏系统设定各类约束条件
- 2) 结合贪心策略求解剩余资金数最大

6.2 模型缺点

- 1) 需要预先求解各个区域之间的无权最短路径

6.3 模型推广

穿越沙漠小游戏与网络上一些游戏玩法较为类似，多名玩家同时推进游戏，玩家之间不同策略对其他玩家有影响，符合当今游戏潮流玩法。本模型适用于大多数同类型的游戏项目优化策略选取。

七、参考文献

[1]百度百科.邻接矩阵

<https://baike.baidu.com/item/%E9%82%BB%E6%8E%A5%E7%9F%A9%E9%98%B5>

[2]龚诗楠. 大规模复杂网络的最短路径近似算法研究[D].电子科技大学,2013.

[3]方挺,王勇,褚文奎,谭小虎.基于贪心思想的调度表优化算法设计[J].计算机工程,2018,44(09):280-285.

[4]陈天鹏,郭雨竹.基于邻接矩阵的行星齿轮系同构判定方法[J].内燃机与配件,2020(12):5-9.

[5]王振忠,霍兴凯,张坚,胡赞.快堆堆芯蒙特卡罗计算模型几何优化问题研究[J/OL]. 原子能科学技术 :1-7[2020-09-13].<http://kns.cnki.net/kcms/detail/11.2044.TL.20200825.1620.010.html>.

附录

一、支撑材料的文件列表

- 1、problem1_1.m
- 2、problem1_2.m
- 3、problem2_1.m
- 4、problem2_2.m
- 5、problem3_1.m
- 6、problem3_2.m
- 7、Result_1.xlsx
- 8、Result_2.xlsx
- 9、Result_3_1.xlsx
- 10、Result_3_2.xlsx
- 11、附件 1：相邻区域矩阵.xlsx
- 12、附件 2：参数设定.xlsx

二、程序代码

1、问题 1 的 *Matlab* 程序代码

第一关程序名：problem1_1.m

```
wen1_1(330, 180);

function [] = wen1_1( foodN, waterN )
    add1 = '附件 2：参数设定.xlsx'; % 信息路径
    init_info(add1);
    global bear_limit;      % 负重上限
    global basic_fund;      % 基础资金
    global water_basic_price; % 水基准价
    global water_consum; % 1：水晴朗消耗 2：水高温消耗 3：水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum; % 1：食物晴朗消耗 2：食物高温消耗 3：食物沙暴消耗
    global weather; % 天气情况
    dis = readmatrix('附件 1：相邻区域矩阵.xlsx', 'Sheet', '第一关', 'Range', 'B3:AB29');
    way = zeros(30, 1); % 0 就是走，1 就是挖，2 是在村庄
```

```

road = [1 25 24 23 23 22 9 9 15 14 12 12 12 12 12 12 12
12 12 14 15 9 21 27

```

```

];

way([11:17]) = 1;
way([8, 20]) = 2;
cost = zeros(numel(road), 2);

for k1 = 2:numel(road)
    day = k1 - 1;
    w = weather(day);
    wc = water_consum(w);
    fc = food_consum(w);
    if way(day) == 0 || way(day) == 2
        % 正常走或停留
        if w == 3
            % 沙暴一倍消耗
            cost(k1, 1) = wc;
            cost(k1, 2) = fc;
        else
            cost(k1, 1) = wc * 2;
            cost(k1, 2) = fc * 2;
        end
    else
        cost(k1, 1) = wc * 3;
        cost(k1, 2) = fc * 3;
    end
end

fund = basic_fund - getM( foodN, waterN, 1 );
bear = bear_limit - getW( foodN, waterN );
food = foodN;
water = waterN;

```

```

spare = zeros(numel(road), 3);
spare(:, 1) = fund;
spare(1, 2) = water;
spare(1, 3) = food;
two = [];
for k1 = 1:(numel(road)-1)
    if way(k1) == 2
        two = [two k1 + 1];
    end
end

need = zeros(numel(road), 2);
if numel(two) == 1
    need(two(1), 1) = sum(cost(two(1):numel(road), 1));
    need(two(1), 2) = sum(cost(two(1):numel(road), 2));
else
    for k1 = 1:(numel(two) - 1)
        need(two(k1), 1) = sum(cost(two(k1):two(k1 + 1), 1));
        need(two(k1), 2) = sum(cost(two(k1):two(k1 + 1), 2));
    end
    need(two(end), 1) = sum(cost(two(end):numel(road), 1));
    need(two(end), 2) = sum(cost(two(end):numel(road), 2));
end

now_fund = fund;

for k1 = 2:numel(road)
    % 剩余的 = 现有的减去消耗的

    if way(k1 - 1) == 2
        if water < need(k1, 1)
            water = water + need(k1, 1);

```



```

        now_fund = now_fund - need(k1, 1) * 2 * water_basic_price;
        while getW( food, water ) > bear_limit
            now_fund = now_fund + 2 * water_basic_price;
            water = water - 1;
        end
    end
    if food < need(k1, 2)
        food = food + need(k1, 2);
        now_fund = now_fund - need(k1, 2) * 2 * food_basic_price;
        while getW( food, water ) > bear_limit
            food = food - 1;
            now_fund = now_fund + 2 * food_basic_price;
        end
    end
end
elseif way(k1 - 1) == 1
    now_fund = now_fund + 1000;
end
spare(k1, 1) = now_fund;
spare(k1, 2:3) = [water, food] - cost(k1, :);
water = water - cost(k1, 1);
food = food - cost(k1, 2);

ak = 0; % 水弥补数
bk = 0; % 食物弥补数
if water < 0
    ak = -water;
end
if bk < 0
    bk = -bk;
end
if ak > 0 || bk > 0
    for kk = k1:-1:1
        spare(kk, 2) = spare(kk, 2) + ak;
        spare(kk, 3) = spare(kk, 3) + bk;
    end
end

```

```

        spare(kk, 1) = spare(kk, 1) - getM(bk, ak, 2);
        if ismember(kk, two)
            break;
        end
    end
end
end

```

```

end

```

```

a = spare(end, 2); % 水多余量
b = spare(end, 3); % 食物多余量
for k1 = numel(road):-1:1
    if spare(k1, 2) ~= 0
        spare(k1, 2) = spare(k1, 2) - a;
    end
    if spare(k1, 3) ~= 0
        spare(k1, 3) = spare(k1, 3) - b;
    end
    spare(k1, 1) = spare(k1, 1) + getM(b, a, 2);
    if ismember(k1, two)
        break;
    end
end
data = [[0:numel(road)-1]' road' spare ];
return ;

```

```

end

```

```

function [w] = getW(food, water)
    global water_weight;
    global food_weight;
    w = water * water_weight + food * food_weight;
end

```

```

function [m] = getM(food, water, c)

```

```

    global water_basic_price;
    global food_basic_price;
    m = water * c * water_basic_price + food * c * food_basic_price;
end

function [] = init_info(add)
    global bear_limit ;      % 负重上限
    global deadline;        % 截止日期
    global basic_fund;       % 基础资金
    global basic_earn;       % 基础收益
    global water_weight;     % 水每箱质量
    global food_weight;      % 食物每箱质量
    global water_basic_price; % 水基准价
    global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
    global weather; % 天气情况
    global adj; % 地图邻接表

    weather_info = readmatrix(add, 'Sheet', '第一关', 'Range', 'H3:Q8', 'OutputType',
'string');
    weather = zeros(30, 1);
    for k1 = 1:2:5
        for k2 = 1:10
            day = str2double(weather_info(k1, k2));
            winfo = weather_info(k1 + 1, k2);
            val = -1;
            switch( winfo )
                case "晴朗"
                    val = 1;
                case "高温"
                    val = 2;
                case "沙暴"
                    val = 3;
            end
        end
    end
end

```

```

        end
        weather(day) = val;
    end
end
bear_limit_text = char(readmatrix(add, 'Sheet', '第一关', 'Range', 'C3:C3', 'OutputType',
'string')); % 负重上限信息
deadline_text = char(readmatrix(add, 'Sheet', '第一关', 'Range', 'C4:C4', 'OutputType',
'string')); % 截止日期信息
basic_fund_text = char(readmatrix(add, 'Sheet', '第一关', 'Range', 'E3:E3', 'OutputType',
'string')); % 基础资金信息
basic_earn_text = char(readmatrix(add, 'Sheet', '第一关', 'Range', 'E4:E4', 'OutputType',
'string')); % 基础收益信息

bear_limit = str2double( bear_limit_text( 1:( strfind( bear_limit_text, '千' ) - 1 ) ) );
deadline = str2double( deadline_text( ( strfind( deadline_text, '第' ) +
1 ):( strfind( deadline_text, '天' ) - 1 ) ) );
basic_fund = str2double( basic_fund_text( 1:( strfind( basic_fund_text, '元' ) - 1 ) ) );
basic_earn = str2double( basic_earn_text( 1:( strfind( basic_earn_text, '元' ) - 1 ) ) );

water_consum = zeros(3, 1);
food_consum = zeros(3, 1);

arr = num2cell(readmatrix(add, 'Sheet', '第一关', 'Range', 'B8:F8'));
[water_weight, water_basic_price, water_consum(1), water_consum(2),
water_consum(3)] = arr{:};

arr = num2cell(readmatrix(add, 'Sheet', '第一关', 'Range', 'B9:F9'));
[food_weight, food_basic_price, food_consum(1), food_consum(2), food_consum(3)] =
arr{:};

dis = readmatrix('附件 1： 相邻区域矩阵.xlsx', 'Sheet', '第一关', 'Range', 'B3:AB29');
for k1 = 1:size(dis, 1)
    dis(k1, k1) = 0;

```

```

end

% 构建邻接表
adj = struct('arr', []);
[r, c] = size(dis);
for k1 = 1:r
    adj(k1).arr = [];
end
for k1 = 1:r
    for k2 = 1:c
        if dis(k1, k2) == 1
            adj(k1).arr = [adj(k1).arr k2];
        end
    end
end
end

clear weather_info day winfo val bear_limit_text deadline_text basic_fund basic_earn
arr dis;
return ;
end

```

第二关程序名：problem1_2.m

```

wen1_2(324, 184);

function [] = wen1_2( foodN, waterN )
    add1 = '附件 2: 参数设定.xlsx'; % 信息路径
    init_info(add1);
    global bear_limit ;      % 负重上限
    global basic_fund;      % 基础资金
    global water_basic_price; % 水基准价
    global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗

```

```

global weather; % 天气情况
dis = readmatrix('附件 1： 相邻区域矩阵.xlsx', 'Sheet', '第二关', 'Range', 'B3:BM66');
way = zeros(30, 1); % 0 就是走， 1 就是挖, 2 是在村庄

road = [1 2 3 4 4 5 13 13 22 30 30 30 39 47 ...
        55 55 55 55 55 62 55 55 55 55 55 55 55 55 55 56 64
];

way([10:11]) = 1;
way([12, 19]) = 2;
way([15:18]) = 1;
way([21:28]) = 1 ;
cost = zeros(numel(road), 2);

for k1 = 2:numel(road)
    day = k1 - 1;
    w = weather(day);
    wc = water_consum(w);
    fc = food_consum(w);
    if way(day) == 0 || way(day) == 2
        % 正常走或停留
        if w == 3
            % 沙暴一倍消耗
            cost(k1, 1) = wc;
            cost(k1, 2) = fc;
        else
            cost(k1, 1) = wc * 2;
            cost(k1, 2) = fc * 2;
        end
    else
        cost(k1, 1) = wc * 3;
        cost(k1, 2) = fc * 3;
    end
end
end

```

```

fund = basic_fund - getM( foodN, waterN, 1 );
bear = bear_limit - getW( foodN, waterN );
food = foodN;
water = waterN;

spare = zeros(numel(road), 3);
spare(:, 1) = fund;
spare(1, 2) = water;
spare(1, 3) = food;
two = [];
for k1 = 1:(numel(road)-1)
    if way(k1) == 2
        two = [two k1 + 1];
    end
end

need = zeros(numel(road), 2);
if numel(two) == 1
    need(two(1), 1) = sum(cost(two(1):numel(road), 1));
    need(two(1), 2) = sum(cost(two(1):numel(road), 2));
else
    for k1 = 1:(numel(two) - 1)
        need(two(k1), 1) = sum(cost(two(k1):two(k1 + 1), 1));
        need(two(k1), 2) = sum(cost(two(k1):two(k1 + 1), 2));
    end
    need(two(end), 1) = sum(cost(two(end):numel(road), 1));
    need(two(end), 2) = sum(cost(two(end):numel(road), 2));
end

now_fund = fund;

for k1 = 2:numel(road)
    % 剩余的 = 现有的减去消耗的

```

```

if way(k1 - 1) == 2
    if water < need(k1, 1)
        water = water + need(k1, 1);
        now_fund = now_fund - need(k1, 1) * 2 * water_basic_price;
        while getW( food, water ) > bear_limit
            now_fund = now_fund + 2 * water_basic_price;
            water = water - 1;
        end
    end
end
if food < need(k1, 2)
    food = food + need(k1, 2);
    now_fund = now_fund - need(k1, 2) * 2 * food_basic_price;
    while getW( food, water ) > bear_limit
        food = food - 1;
        now_fund = now_fund + 2 * food_basic_price;
    end
end
elseif way(k1 - 1) == 1
    now_fund = now_fund + 1000;
end
spare(k1, 1) = now_fund;
spare(k1, 2:3) = [water, food] - cost(k1, :);
water = water - cost(k1, 1);
food = food - cost(k1, 2);

ak = 0; % 水弥补数
bk = 0; % 食物弥补数
if water < 0
    ak = -water;
end
if bk < 0
    bk = -bk;
end
end

```



```

        if ak > 0 || bk > 0
            for kk = k1:-1:1
                spare(kk, 2) = spare(kk, 2) + ak;
                spare(kk, 3) = spare(kk, 3) + bk;
                spare(kk, 1) = spare(kk, 1) - getM(bk, ak, 2);
                if ismember(kk, two)
                    break;
                end
            end
        end
    end

end

a = spare(end, 2); % 水多余量
b = spare(end, 3); % 食物多余量
for k1 = numel(road):-1:1
    if spare(k1, 2) ~= 0
        spare(k1, 2) = spare(k1, 2) - a;
    end
    if spare(k1, 3) ~= 0
        spare(k1, 3) = spare(k1, 3) - b;
    end
    spare(k1, 1) = spare(k1, 1) + getM(b, a, 2);
    if ismember(k1, two)
        break;
    end
end

data = [[0:numel(road)-1]' road' spare ];
return ;

end

function [w] = getW(food, water)
    global water_weight;
    global food_weight;

```

```

        w = water * water_weight + food * food_weight;
    end

function [m] = getM(food, water, c)
    global water_basic_price;
    global food_basic_price;
    m = water * c * water_basic_price + food * c * food_basic_price;
end

function [] = init_info(add)
    global bear_limit ;      % 负重上限
    global deadline;        % 截止日期
    global basic_fund;       % 基础资金
    global basic_earn;       % 基础收益
    global water_weight;     % 水每箱质量
    global food_weight;      % 食物每箱质量
    global water_basic_price; % 水基准价
    global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
    global weather; % 天气情况
    global adj; % 地图邻接表

    weather_info = readmatrix(add, 'Sheet', '第二关', 'Range', 'H3:Q8', 'OutputType',
'string');
    weather = zeros(30, 1);
    for k1 = 1:2:5
        for k2 = 1:10
            day = str2double(weather_info(k1, k2));
            winfo = weather_info(k1 + 1, k2);
            val = -1;
            switch( winfo )
                case "晴朗"
                    val = 1;

```

```

        case "高温"
            val = 2;
        case "沙暴"
            val = 3;
        end
        weather(day) = val;
    end
end

bear_limit_text = char(readmatrix(add, 'Sheet', '第二关', 'Range', 'C3:C3', 'OutputType',
'string')); % 负重上限信息
deadline_text = char(readmatrix(add, 'Sheet', '第二关', 'Range', 'C4:C4', 'OutputType',
'string')); % 截止日期信息
basic_fund_text = char(readmatrix(add, 'Sheet', '第二关', 'Range', 'E3:E3', 'OutputType',
'string')); % 基础资金信息
basic_earn_text = char(readmatrix(add, 'Sheet', '第二关', 'Range', 'E4:E4', 'OutputType',
'string')); % 基础收益信息

bear_limit = str2double( bear_limit_text( 1:( strfind( bear_limit_text, '千' ) - 1 ) ) );
deadline = str2double( deadline_text( ( strfind( deadline_text, '第' ) +
1 ):( strfind( deadline_text, '天' ) - 1 ) ) );
basic_fund = str2double( basic_fund_text( 1:( strfind( basic_fund_text, '元' ) - 1 ) ) );
basic_earn = str2double( basic_earn_text( 1:( strfind( basic_earn_text, '元' ) - 1 ) ) );

water_consum = zeros(3, 1);
food_consum = zeros(3, 1);

arr = num2cell(readmatrix(add, 'Sheet', '第二关', 'Range', 'B8:F8'));
[water_weight, water_basic_price, water_consum(1), water_consum(2),
water_consum(3)] = arr{:};

arr = num2cell(readmatrix(add, 'Sheet', '第二关', 'Range', 'B9:F9'));
[food_weight, food_basic_price, food_consum(1), food_consum(2), food_consum(3)] =
arr{:};

```

```

dis = readmatrix('附件 1： 相邻区域矩阵.xlsx', 'Sheet', '第二关', 'Range', 'B3:BM66');
for k1 = 1:size(dis, 1)
    dis(k1, k1) = 0;
end

% 构建邻接表
adj = struct('arr', []);
[r, c] = size(dis);
for k1 = 1:r
    adj(k1).arr = [];
end
for k1 = 1:r
    for k2 = 1:c
        if dis(k1, k2) == 1
            adj(k1).arr = [adj(k1).arr k2];
        end
    end
end
end

clear weather_info day winfo val bear_limit_text deadline_text basic_fund basic_earn
arr dis;
return ;
end

```

2、问题 2 的 *Matlab* 程序代码

第三关程序名： **problem2_1.m**

```

wen2_2(54, 54);
function [] = wen2_2( foodN, waterN )
    add1 = '附件 2： 参数设定.xlsx'; % 信息路径
    init_info(add1);
    global bear_limit ;      % 负重上限
    global basic_fund;      % 基础资金

```

```

global basic_earn;      % 基础收益
global water_basic_price; % 水基准价
global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
global food_basic_price; % 食物基准价
global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
global weather; % 天气情况

way = zeros(30, 1); % 0 就是走, 1 就是挖, 2 是在村庄

road = [1 4 6 13];
cost = zeros(numel(road), 2);

for k1 = 2:numel(road)
    day = k1 - 1;
    w = weather(day);
    wc = water_consum(w);
    fc = food_consum(w);
    if way(day) == 0 || way(day) == 2
        % 正常走或停留
        if w == 3
            % 沙暴一倍消耗
            cost(k1, 1) = wc;
            cost(k1, 2) = fc;
        else
            cost(k1, 1) = wc * 2;
            cost(k1, 2) = fc * 2;
        end
    else
        cost(k1, 1) = wc * 3;
        cost(k1, 2) = fc * 3;
    end
end

fund = basic_fund - getM( foodN, waterN, 1 );
bear = bear_limit - getW( foodN, waterN );

```

```

food = foodN;
water = waterN;

spare = zeros(numel(road), 3);
spare(:, 1) = fund;
spare(1, 2) = water;
spare(1, 3) = food;
two = [];
for k1 = 1:(numel(road)-1)
    if way(k1) == 2
        two = [two k1 + 1];
    end
end

need = zeros(numel(road), 2);
if numel(two) > 0
    if numel(two) == 1
        need(two(1), 1) = sum(cost(two(1):numel(road), 1));
        need(two(1), 2) = sum(cost(two(1):numel(road), 2));
    else
        for k1 = 1:(numel(two) - 1)
            need(two(k1), 1) = sum(cost(two(k1):two(k1 + 1), 1));
            need(two(k1), 2) = sum(cost(two(k1):two(k1 + 1), 2));
        end
        need(two(end), 1) = sum(cost(two(end):numel(road), 1));
        need(two(end), 2) = sum(cost(two(end):numel(road), 2));
    end
end

now_fund = fund;

for k1 = 2:numel(road)
    % 剩余的 = 现有的减去消耗的

```

```

if way(k1 - 1) == 2
    if water < need(k1, 1)
        water = water + need(k1, 1);
        now_fund = now_fund - need(k1, 1) * 2 * water_basic_price;
        while getW( food, water ) > bear_limit
            now_fund = now_fund + 2 * water_basic_price;
            water = water - 1;
        end
    end
    if food < need(k1, 2)
        food = food + need(k1, 2);
        now_fund = now_fund - need(k1, 2) * 2 * food_basic_price;
        while getW( food, water ) > bear_limit
            food = food - 1;
            now_fund = now_fund + 2 * food_basic_price;
        end
    end
elseif way(k1 - 1) == 1
    now_fund = now_fund + basic_earn;
end
spare(k1, 1) = now_fund;
spare(k1, 2:3) = [water, food] - cost(k1, :);
water = water - cost(k1, 1);
food = food - cost(k1, 2);

ak = 0; % 水弥补数
bk = 0; % 食物弥补数
if water < 0
    ak = -water;
end
if food < 0
    bk = -food;
end
if ak > 0 || bk > 0

```

```

        for kk = k1:-1:1
            spare(kk, 2) = spare(kk, 2) + ak;
            spare(kk, 3) = spare(kk, 3) + bk;
            spare(kk, 1) = spare(kk, 1) - getM(bk, ak, 2);
            if ismember(kk, two)
                break;
            end
        end
    end
end

end

% 第三间出现多余的时候
% 在终点按半价换钱
a = spare(end, 2); % 水多余量
b = spare(end, 3); % 食物多余量
spare(end, 2) = spare(end, 2) - a;
spare(end, 3) = spare(end, 3) - b;
spare(end, 1) = spare(end, 1) + getM(b, a, 0.5);

data = [[0:numel(road)-1]' road' spare ];
return ;
end

function [w] = getW(food, water)
    global water_weight;
    global food_weight;
    w = water * water_weight + food * food_weight;
end

function [m] = getM(food, water, c)
    global water_basic_price;
    global food_basic_price;
    m = water * c * water_basic_price + food * c * food_basic_price;
end

```



```

function [] = init_info(add)
    global bear_limit;    % 负重上限
    global deadline;      % 截止日期
    global basic_fund;    % 基础资金
    global basic_earn;    % 基础收益
    global water_weight;  % 水每箱质量
    global food_weight;   % 食物每箱质量
    global water_basic_price; % 水基准价
    global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
    global weather; % 天气情况

    % 一般天气
    weather_info = readmatrix(add, 'Sheet', '第二关', 'Range', 'H3:Q4', 'OutputType',
'string');
    weather = zeros(10, 1);
    for k1 = 1:1
        for k2 = 1:10
            day = str2double(weather_info(k1, k2));
            winfo = weather_info(k1 + 1, k2);
            val = -1;
            switch( winfo )
                case "晴朗"
                    val = 1;
                case "高温"
                    val = 2;
                case "沙暴"
                    val = 3;
            end
            weather(day) = val;
        end
    end
end

```

```

        bear_limit_text = char(readmatrix(add, 'Sheet', '第三关', 'Range', 'C3:C3', 'OutputType',
'string')); % 负重上限信息

        deadline_text = char(readmatrix(add, 'Sheet', '第三关', 'Range', 'C4:C4', 'OutputType',
'string')); % 截止日期信息

        basic_fund_text = char(readmatrix(add, 'Sheet', '第三关', 'Range', 'E3:E3', 'OutputType',
'string')); % 基础资金信息

        basic_earn_text = char(readmatrix(add, 'Sheet', '第三关', 'Range', 'E4:E4', 'OutputType',
'string')); % 基础收益信息


        bear_limit = str2double( bear_limit_text( 1:( strfind( bear_limit_text, '千' ) - 1 ) ) );
        deadline = str2double( deadline_text( ( strfind( deadline_text, '第' ) +
1 ):( strfind( deadline_text, '天' ) - 1 ) ) );
        basic_fund = str2double( basic_fund_text( 1:( strfind( basic_fund_text, '元' ) - 1 ) ) );
        basic_earn = str2double( basic_earn_text( 1:( strfind( basic_earn_text, '元' ) - 1 ) ) );


        water_consum = zeros(3, 1);
        food_consum = zeros(3, 1);
        arr = num2cell(readmatrix(add, 'Sheet', '第三关', 'Range', 'B8:F8'));
        [water_weight, water_basic_price, water_consum(1), water_consum(2),
water_consum(3)] = arr{:};


        arr = num2cell(readmatrix(add, 'Sheet', '第三关', 'Range', 'B9:F9'));
        [food_weight, food_basic_price, food_consum(1), food_consum(2), food_consum(3)] =
arr{:};


        clear weather_info day winfo val bear_limit_text deadline_text basic_fund basic_earn
arr dis;

        return ;

    end

```

第四关程序名：problem2_2.m

```

wen2_2(240, 240);
function [] = wen2_2( foodN, waterN )

```

```

add1 = '第四关天气情况.xlsx'; % 信息路径
init_info(add1);
global bear_limit; % 负重上限
global basic_fund; % 基础资金
global basic_earn; % 基础收益
global water_basic_price; % 水基准价
global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
global food_basic_price; % 食物基准价
global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
global weather; % 天气情况
way = zeros(30, 1); % 0 就是走, 1 就是挖, 2 是在村庄

road = [1 2 3 8 13 18 18 18 18 19 14 13 18 18 18 18 18
18 19 20 25
];
way(6:8) = 1;
way(13:17) = 1;
way([10]) = 2;
% road = [1 2 3 8 13 18 18 18 18 18 18 18 18 18 18 18
18 19 14 13 18 18 18 18 18 18 19 20 25
% ];
% way([6:16]) = 1;
% way([21:26]) = 1;
% way([18]) = 2;
% road = [1 2 3 8 13 18 18 18 18 18 18 18 19 14 13 18
18 18 18 18 18 18 18 19 14 15 20 25
% ];
% way(6:11) = 1;
% way(16:22) = 1;
% way([13, 24]) = 2;
cost = zeros(numel(road), 2);

for k1 = 2:numel(road)
    day = k1 - 1;

```

```

w = weather(day);
wc = water_consum(w);
fc = food_consum(w);
if way(day) == 0 || way(day) == 2
    % 正常走或停留
    if w == 3
        % 沙暴一倍消耗
        cost(k1, 1) = wc;
        cost(k1, 2) = fc;
    else
        cost(k1, 1) = wc * 2;
        cost(k1, 2) = fc * 2;
    end
else
    cost(k1, 1) = wc * 3;
    cost(k1, 2) = fc * 3;
end
end

fund = basic_fund - getM( foodN, waterN, 1 );
bear = bear_limit - getW( foodN, waterN );
food = foodN;
water = waterN;

spare = zeros(numel(road), 3);
spare(:, 1) = fund;
spare(1, 2) = water;
spare(1, 3) = food;
two = [];
for k1 = 1:(numel(road)-1)
    if way(k1) == 2
        two = [two k1 + 1];
    end
end
end

```

```

need = zeros(numel(road), 2);
if numel(two) > 0
    if numel(two) == 1
        need(two(1), 1) = sum(cost(two(1):numel(road), 1));
        need(two(1), 2) = sum(cost(two(1):numel(road), 2));
    else
        for k1 = 1:(numel(two) - 1)
            need(two(k1), 1) = sum(cost(two(k1):two(k1 + 1), 1));
            need(two(k1), 2) = sum(cost(two(k1):two(k1 + 1), 2));
        end
        need(two(end), 1) = sum(cost(two(end):numel(road), 1));
        need(two(end), 2) = sum(cost(two(end):numel(road), 2));
    end
end

now_fund = fund;

for k1 = 2:numel(road)
    % 剩余的 = 现有的减去消耗的

    if way(k1 - 1) == 2
        if water < need(k1, 1) && food < need(k1, 2)
            while water < need(k1, 1) && food < need(k1, 2)
                water = water + need(k1, 1);
                food = food + need(k1, 2);
                now_fund = now_fund - getM(need(k1, 2), need(k1, 1), 2);
            end
            while getW( food, water ) > bear_limit
                now_fund = now_fund + getM(1, 1, 2);
                water = water - 1;
                food = food - 1;
            end
        end
    end
end

```

```

if water < need(k1, 1)
    water = water + need(k1, 1);
    now_fund = now_fund - need(k1, 1) * 2 * water_basic_price;
    while getW( food, water ) > bear_limit
        now_fund = now_fund + 2 * water_basic_price;
        water = water - 1;
    end
end
if food < need(k1, 2)
    food = food + need(k1, 2);
    now_fund = now_fund - need(k1, 2) * 2 * food_basic_price;
    while getW( food, water ) > bear_limit
        food = food - 1;
        now_fund = now_fund + 2 * food_basic_price;
    end
end
elseif way(k1 - 1) == 1
    now_fund = now_fund + basic_earn;
end
spare(k1, 1) = now_fund;
spare(k1, 2:3) = [water, food] - cost(k1, :);
water = water - cost(k1, 1);
food = food - cost(k1, 2);

ak = 0; % 水弥补数
bk = 0; % 食物弥补数
if water < 0
    ak = -water;
end
if food < 0
    bk = -food;
end
if ak > 0 || bk > 0
    for kk = k1:-1:1

```

```

        spare(kk, 2) = spare(kk, 2) + ak;
        spare(kk, 3) = spare(kk, 3) + bk;
        spare(kk, 1) = spare(kk, 1) - getM(bk, ak, 2);
        if ismember(kk, two)
            break;
        end
    end
end

end

% 在终点按半价换钱
a = spare(end, 2); % 水多余量
b = spare(end, 3); % 食物多余量
spare(end, 2) = spare(end, 2) - a;
spare(end, 3) = spare(end, 3) - b;
spare(end, 1) = spare(end, 1) + getM(b, a, 0.5);

data = [[0:numel(road)-1]' road' spare ];
return ;
end

function [w] = getW(food, water)
    global water_weight;
    global food_weight;
    w = water * water_weight + food * food_weight;
end

function [m] = getM(food, water, c)
    global water_basic_price;
    global food_basic_price;
    m = water * c * water_basic_price + food * c * food_basic_price;
end

function [] = init_info(add)

```

```

global bear_limit ;      % 负重上限
global deadline;        % 截止日期
global basic_fund;      % 基础资金
global basic_earn;      % 基础收益
global water_weight;    % 水每箱质量
global food_weight;     % 食物每箱质量
global water_basic_price; % 水基准价
global water_consum;    % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
global food_basic_price; % 食物基准价
global food_consum;     % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
global weather; % 天气情况

%      % 晴朗极端天气
%      weather_info = readmatrix(add, 'Range', 'A16:J21', 'OutputType', 'string');

%      % 高温极端天气
%      weather_info = readmatrix(add, 'Range', 'A28:J33', 'OutputType', 'string');
%
% 一般天气
weather_info = readmatrix(add, 'Range', 'A3:J8', 'OutputType', 'string');
weather = zeros(30, 1);
for k1 = 1:2:5
    for k2 = 1:10
        day = str2double(weather_info(k1, k2));
        winfo = weather_info(k1 + 1, k2);
        val = -1;
        switch( winfo )
            case "晴朗"
                val = 1;
            case "高温"
                val = 2;
            case "沙暴"
                val = 3;
        end
    end
end

```



```

        weather(day) = val;
    end
end
add2 = '附件 2: 参数设定.xlsx';
bear_limit_text = char(readmatrix(add2, 'Sheet', '第四关', 'Range', 'C3:C3', 'OutputType',
'string')); % 负重上限信息
deadline_text = char(readmatrix(add2, 'Sheet', '第四关', 'Range', 'C4:C4', 'OutputType',
'string')); % 截止日期信息
basic_fund_text = char(readmatrix(add2, 'Sheet', '第四关', 'Range', 'E3:E3', 'OutputType',
'string')); % 基础资金信息
basic_earn_text = char(readmatrix(add2, 'Sheet', '第四关', 'Range', 'E4:E4', 'OutputType',
'string')); % 基础收益信息

bear_limit = str2double( bear_limit_text( 1:( strfind( bear_limit_text, '千' ) - 1 ) ) );
deadline = str2double( deadline_text( ( strfind( deadline_text, '第' ) +
1 ):( strfind( deadline_text, '天' ) - 1 ) ) );
basic_fund = str2double( basic_fund_text( 1:( strfind( basic_fund_text, '元' ) - 1 ) ) );
basic_earn = str2double( basic_earn_text( 1:( strfind( basic_earn_text, '元' ) - 1 ) ) );

water_consum = zeros(3, 1);
food_consum = zeros(3, 1);

arr = num2cell(readmatrix(add2, 'Sheet', '第四关', 'Range', 'B8:F8'));
[water_weight, water_basic_price, water_consum(1), water_consum(2),
water_consum(3)] = arr{:};

arr = num2cell(readmatrix(add2, 'Sheet', '第四关', 'Range', 'B9:F9'));
[food_weight, food_basic_price, food_consum(1), food_consum(2), food_consum(3)] =
arr{:};

clear weather_info day winfo val bear_limit_text deadline_text basic_fund basic_earn
arr dis;

return ;

```

end

3、问题 3 的 *Matlab* 程序代码

第五关程序名： **problem3_1.m**

```
wen3_1(42, 36, 42, 36);  
function [] = wen3_1( foodN1, waterN1, foodN2, waterN2 )  
    add1 = '附件 2: 参数设定.xlsx'; % 信息路径  
    init_info(add1);  
    global bear_limit ;      % 负重上限  
    global basic_fund;      % 基础资金  
    global basic_earn;      % 基础收益  
    global water_basic_price; % 水基准价  
    global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗  
    global food_basic_price; % 食物基准价  
    global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗  
    global weather; % 天气情况  
    % way = zeros(30, 1); % 0 就是走, 1 就是挖, 2 是在村庄  
  
    roads = struct('road', [], 'way', []);  
    roads(1).road = [1 5 6 13];  
    roads(2).road = [1 4 6 13];  
  
    roads(1).way = zeros(30, 1);  
    roads(2).way = zeros(30, 1);  
  
    cost = zeros(30, 4);  
  
    position = zeros(30, 30); % 记录某位置玩家数量  
  
    for k1 = 1:numel(roads)  
        % 每一个玩家  
        arr = roads(k1).road;  
        for k2 = 1:numel(arr) % 记录位置
```

```

        k = arr(k2);
        position(k2, k) = position(k2, k) + 1;
    end
end

for k2 = 1:numel(roads)
    col = (k2 - 1) * 2 + 1;
    for k1 = 2:numel(roads(k2).road)
        day = k1 - 1;
        w = weather(day);
        wc = water_consum(w);
        fc = food_consum(w);
        player_pos = roads(k2).road(k1);
        if roads(k2).way(day) == 0 || roads(k2).way(day) == 2
            % 正常走
            if w == 3
                % 沙暴停留
                cost(k1, col) = wc;
                cost(k1, col + 1) = fc;
            else
                % 行走
                try
                    if position(day, roads(k2).road(day)) == 2 && position(day + 1,
roads(k2).road(day + 1)) == 2
                        % 两人走同样的线路
                        k = 2;
                    else
                        k = 1;
                    end
                catch
                    k = 1;
                end
            end
            cost(k1, col) = wc * 2 * k;
            cost(k1, col + 1) = fc * 2 * k;
        end
    end
end

```

```

        end
    else
        % 挖矿
        cost(k1, col) = wc * 3;
        cost(k1, col + 1) = fc * 3;
    end
end
end
end

fund = zeros(2, 1);
bear = zeros(2, 1);
food = zeros(2, 1);
water = zeros(2, 1);

fund(1) = basic_fund - getM( foodN1, waterN1, 1 );
bear(1) = bear_limit - getW( foodN1, waterN1 );

fund(2) = basic_fund - getM( foodN2, waterN2, 1 );
bear(2) = bear_limit - getW( foodN2, waterN2 );

food(1) = foodN1;
water(1) = waterN1;
food(2) = foodN2;
water(2) = waterN2;

spare = zeros(30, 6);
spare(:, 1) = fund(1);
spare(1, 2) = water(1);
spare(1, 3) = food(1);

spare(:, 4) = fund(2);
spare(1, 5) = water(2);
spare(1, 6) = food(2);

```

```

twos = struct('two', []);
for k1 = 1:numel(roads)
    twos(k1).two = [];
end

for k1 = 1:numel(roads)
    for k2 = 1:(numel(roads(k1).road) - 1)
        if roads(k1).way(k2) == 2
            twos(k1).two = [twos(k1).two k2 + 1];
        end
    end
end

need = zeros(30, 4);
for k1 = 1:numel(roads)
    pos = (k1 - 1) * 2 + 1;
    if numel(twos(k1).two) > 0
        if numel(twos(k1).two) == 1
            need(twos(k1).two(1), pos) =
sum(cost(twos(k1).two(1):numel(roads(k1).road), pos));
            need(twos(k1).two(1), pos + 1) =
sum(cost(twos(k1).two(1):numel(roads(k1).road), pos + 1));
        else
            for k3 = 1:(numel(twos(k1).two) - 1)
                need(twos(k1).two(k3), pos) =
sum(cost(twos(k1).two(k3):twos(k1).two(k3 + 1), pos));
                need(twos(k1).two(k3), pos + 1) =
sum(cost(twos(k1).two(k3):twos(k1).two(k3 + 1), pos + 1));
            end
            need(twos(k1).two(end), pos) =
sum(cost(twos(k1).two(end):numel(roads(k1).road), pos));
            need(twos(k1).two(end), pos + 1) =
sum(cost(twos(k1).two(end):numel(roads(k1).road), pos));
        end
    end
end

```

```

        end
    end

    now_fund = fund;

    last_rate = 2;
    for k2 = 1:numel(roads)
        pos = (k2 - 1) * 2 + 1;
        for k1 = 2:numel(roads(k2).road)
            % 剩余的 = 现有的减去消耗的

            if roads(k2).way(k1 - 1) == 2
                % 村庄补给
                % 第 k1 天第 road(k1)个区域的人数
                rate = 2;
                if position(k1, roads(k2).road(k1)) > 1
                    rate = 4;
                end
                last_rate = rate;
                % k2 人数， k1 天数
                if water(k2) < need(k1, pos) && food < need(k1, pos + 1)
                    water(k2) = water(k2) + need(k1, pos);
                    food(k2) = food(k2) + need(k1, pos + 1);

                    now_fund(k2) = now_fund(k2) - getM(need(k1, pos + 1), need(k1,
pos), rate);

                    while getW( food, water ) > bear_limit
                        now_fund(k2) = now_fund(k2) + getM(1, 1, 2);
                        water(k2) = water(k2) - 1;
                        food(k2) = food(k2) - 1;
                    end
                end
            end
        end
    end

```

```

end

if water(k2) < need(k1, pos)
    water(k2) = water(k2) + need(k1, pos);

    now_fund(k2) = now_fund(k2) - need(k1, pos) * rate *
water_basic_price;

    while getW( food(k2), water(k2) ) > bear_limit
        now_fund(k2) = now_fund(k2) + rate * water_basic_price;
        water(k2) = water(k2) - 1;
    end
end

if food(k2) < need(k1, pos + 1)
    food(k2) = food(k2) + need(k1, pos + 1);
    now_fund(k2) = now_fund(k2) - need(k1, pos + 1) * rate *
food_basic_price;

    while getW( food(k2), water(k2) ) > bear_limit
        food(k2) = food(k2) - 1;
        now_fund(k2) = now_fund(k2) + rate * food_basic_price;
    end
end

elseif roads(k2).way(k1 - 1) == 1
    % k2 人数， k1 天数
    rate = 1; % 初始倍数是 1
    if position(k1, roads(k2).road(k1)) > 1
        % 如果多个人挖，倍数为 1 / 人数
        rate = 1 / position(k1, roads(k2).road(k1));
    end
    now_fund(k2) = now_fund(k2) + rate * basic_earn;
end

% 结果矩阵有 3 列，要针对人数选择不同的结果列起点，一个人的结果
是 1-3，两个人是 4-6
pos2 = (k2 - 1) * 3 + 1;

```

```

    spare(k1, pos2) = now_fund(k2);
    spare(k1, (pos2 + 1):(pos2 + 2) ) = [water(k2), food(k2)] - cost(k1, pos:(pos +
1));

    water(k2) = water(k2) - cost(k1, pos);
    food(k2) = food(k2) - cost(k1, pos + 1);

    ak = 0; % 水弥补数
    bk = 0; % 食物弥补数
    if water(k2) < 0
        ak = -water(k2);
    end
    if bk < 0
        bk = -food(k2);
    end
    if ak > 0 || bk > 0
        for kk = k1:-1:1
            spare(kk, pos2 + 1) = spare(kk, pos2 + 1) + ak;
            spare(kk, pos2 + 2) = spare(kk, pos2 + 2) + bk;
            spare(kk, pos2) = spare(kk, pos2) - getM(bk, ak, last_rate);
            if ismember(kk, twos(k2).two)
                break;
            end
        end
    end

end

end

for k2 = 1:numel(roads)
    pos2 = (k2 - 1) * 3 + 1;
    a = spare(end, pos2 + 1); % 水多余量
    b = spare(end, pos2 + 2); % 食物多余量
    for k1 = numel(roads(k2).road):-1:1
        if spare(k1, pos2 + 1) ~= 0

```



```

        spare(k1, pos2 + 1) = spare(k1, pos2 + 1) - a;
    end
    if spare(k1, pos2 + 2) ~= 0
        spare(k1, pos2 + 2) = spare(k1, pos2 + 2) - b;
    end
    spare(k1, pos2) = spare(k1, pos2) + getM(b, a, last_rate);
    if ismember(k1, twos(k2).two)
        break;
    end
end
end
end

```

```

size1 = numel(roads(1).road);
size2 = numel(roads(2).road);
data = zeros(max(size1, size2), 10);
data(1:size1, 1:5) = [[0:size1-1]' roads(1).road' spare(1:size1, 1:3) ];
data(1:size2, 6:10) = [[[0:size2-1]' roads(2).road' spare(1:size2, 4:6)] ];

```

```

    return ;
end

```

```

function [w] = getW(food, water)
    global water_weight;
    global food_weight;
    w = water * water_weight + food * food_weight;
end

```

```

function [m] = getM(food, water, c)
    global water_basic_price;
    global food_basic_price;
    m = water * c * water_basic_price + food * c * food_basic_price;
end

```

```

function [] = init_info(add)

    global bear_limit ;      % 负重上限
    global deadline;        % 截止日期
    global basic_fund;       % 基础资金
    global basic_earn;       % 基础收益
    global water_weight;     % 水每箱质量
    global food_weight;      % 食物每箱质量
    global water_basic_price; % 水基准价
    global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
    global weather; % 天气情况

    % 一般天气

    weather_info = readmatrix(add, 'Sheet', '第五关', 'Range', 'H3:Q4', 'OutputType',
'string');

    weather = zeros(10, 1);

    for k1 = 1:1
        for k2 = 1:10
            day = str2double(weather_info(k1, k2));
            winfo = weather_info(k1 + 1, k2);
            val = -1;
            switch( winfo )
                case "晴朗"
                    val = 1;
                case "高温"
                    val = 2;
                case "沙暴"
                    val = 3;
            end
            weather(day) = val;
        end
    end

    bear_limit_text = char(readmatrix(add, 'Sheet', '第五关', 'Range', 'C3:C3', 'OutputType',

```

```

'string')); % 负重上限信息
    deadline_text = char(readmatrix(add, 'Sheet', '第五关', 'Range', 'C4:C4', 'OutputType',
'string')); % 截止日期信息
    basic_fund_text = char(readmatrix(add, 'Sheet', '第五关', 'Range', 'E3:E3', 'OutputType',
'string')); % 基础资金信息
    basic_earn_text = char(readmatrix(add, 'Sheet', '第五关', 'Range', 'E4:E4', 'OutputType',
'string')); % 基础收益信息

    bear_limit = str2double( bear_limit_text( 1:( strfind( bear_limit_text, '千' ) - 1 ) ) );
    deadline = str2double( deadline_text( ( strfind( deadline_text, '第' ) +
1 ):( strfind( deadline_text, '天' ) - 1 ) ) );
    basic_fund = str2double( basic_fund_text( 1:( strfind( basic_fund_text, '元' ) - 1 ) ) );
    basic_earn = str2double( basic_earn_text( 1:( strfind( basic_earn_text, '元' ) - 1 ) ) );

    water_consum = zeros(3, 1);
    food_consum = zeros(3, 1);

    arr = num2cell(readmatrix(add, 'Sheet', '第五关', 'Range', 'B8:F8'));
    [water_weight, water_basic_price, water_consum(1), water_consum(2),
water_consum(3)] = arr{:};

    arr = num2cell(readmatrix(add, 'Sheet', '第五关', 'Range', 'B9:F9'));
    [food_weight, food_basic_price, food_consum(1), food_consum(2), food_consum(3)] =
arr{:};
    clear weather_info day winfo val bear_limit_text deadline_text basic_fund basic_earn
arr dis;
    return ;
end

```

第六关程序名：problem3_2.m

```

wen3_2(240, 150, 240, 240, 240, 240);
function [] = wen3_2( foodN1, waterN1, foodN2, waterN2, foodN3, waterN3 )
    add1 = '附件 2: 参数设定.xlsx'; % 信息路径

```

```

init_info(add1);
global bear_limit;      % 负重上限
global basic_fund;      % 基础资金
global basic_earn;      % 基础收益
global water_basic_price; % 水基准价
global water_consum; % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
global food_basic_price; % 食物基准价
global food_consum; % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
global weather; % 天气情况
% way = zeros(30, 1); % 0 就是走, 1 就是挖, 2 是在村庄

roads = struct('road', [], 'way', []);
roads(1).road = [1 6 11 16 17 18 18 18 18 18 13 14 19 18 18
18 18 18 18 18 18 19 20 25
];
roads(2).road = [1 2 7 12 13 18 18 13 14 19 18 18 18 18 18
18 18 18 19 20 25
];
roads(3).road = [1 2 3 4 9 14 19 18 18 18 18 18 18 18 18
19 20 20 25
];

roads(1).way = zeros(30, 1);
roads(2).way = zeros(30, 1);
roads(3).way = zeros(30, 1);

roads(1).way([6:9]) = 1;
roads(1).way([14:20]) = 1;
roads(1).way([11]) = 2;

roads(2).way([6]) = 1;
roads(2).way([11:16]) = 1;
roads(2).way([8]) = 2;

```

```

roads(3).way([5]) = 2;
roads(3).way([8:14]) = 1;

cost = zeros(30, 6);

position = zeros(30, 30); % 记录某位置玩家数量

for k1 = 1:numel(roads)
    % 每一个玩家
    arr = roads(k1).road;
    for k2 = 1:numel(arr) % 记录位置
        k = arr(k2);
        position(k2, k) = position(k2, k) + 1;
    end
end

for k2 = 1:numel(roads)
    col = (k2 - 1) * 2 + 1;
    for k1 = 2:numel(roads(k2).road)
        day = k1 - 1;
        w = weather(day);
        wc = water_consum(w);
        fc = food_consum(w);

        if roads(k2).way(day) == 0 || roads(k2).way(day) == 2
            % 正常走
            if w == 3
                % 沙暴停留
                cost(k1, col) = wc;
                cost(k1, col + 1) = fc;
            else
                % 行走

```

```

k = 1;
another = setdiff(1:3, k2); % 得到其余玩家
try
    id = another(1); % 得到第一位玩家 ID
    player1_begin = roads(k2).road(day); %第一位玩家位置
    player1_end = roads(k2).road(day + 1);

    player2_begin = roads(id).road(day);
    player2_end = roads(id).road(day + 1);

    if player1_begin == player2_begin && player1_end ==
player2_end

        k = k + 1;
    end
catch
end

try
    id = another(2); % 得到第二位玩家 ID
    player1_begin = roads(k2).road(day); %第二位玩家位置
    player1_end = roads(k2).road(day + 1);

    player2_begin = roads(id).road(day);
    player2_end = roads(id).road(day + 1);

    if player1_begin == player2_begin && player1_end ==
player2_end

        k = k + 1;
    end
catch
end
cost(k1, col) = wc * 2 * k;
cost(k1, col + 1) = fc * 2 * k;
end

```

```

        else
            % 挖矿
            cost(k1, col) = wc * 3;
            cost(k1, col + 1) = fc * 3;
        end
    end
end

fund = zeros(3, 1);
bear = zeros(3, 1);
food = zeros(3, 1);
water = zeros(3, 1);

fund(1) = basic_fund - getM( foodN1, waterN1, 1 );
bear(1) = bear_limit - getW( foodN1, waterN1 );

fund(2) = basic_fund - getM( foodN2, waterN2, 1 );
bear(2) = bear_limit - getW( foodN2, waterN2 );

fund(3) = basic_fund - getM( foodN3, waterN3, 1 );
bear(3) = bear_limit - getW( foodN3, waterN3 );

food(1) = foodN1;
water(1) = waterN1;
food(2) = foodN2;
water(2) = waterN2;
food(3) = foodN3;
water(3) = waterN3;

spare = zeros(30, 9);
spare(:, 1) = fund(1);
spare(1, 2) = water(1);
spare(1, 3) = food(1);

```

```

spare(:, 4) = fund(2);
spare(1, 5) = water(2);
spare(1, 6) = food(2);

spare(:, 7) = fund(3);
spare(1, 8) = water(3);
spare(1, 9) = food(3);

twos = struct('two', []);
for k1 = 1:numel(roads)
    twos(k1).two = [];
end

for k1 = 1:numel(roads)
    for k2 = 1:(numel(roads(k1).road) - 1)
        if roads(k1).way(k2) == 2
            twos(k1).two = [twos(k1).two k2 + 1];
        end
    end
end

need = zeros(30, 6);
for k1 = 1:numel(roads)
    pos = (k1 - 1) * 2 + 1;
    if numel(twos(k1).two) > 0
        if numel(twos(k1).two) == 1
            need(twos(k1).two(1), pos) =
sum(cost(twos(k1).two(1):numel(roads(k1).road), pos));
            need(twos(k1).two(1), pos + 1) =
sum(cost(twos(k1).two(1):numel(roads(k1).road), pos + 1));
        else
            for k3 = 1:(numel(twos(k1).two) - 1)
                need(twos(k1).two(k3), pos) =
sum(cost(twos(k1).two(k3):twos(k1).two(k3 + 1), pos));
            end
        end
    end
end

```



```

                                need(twos(k1).two(k3), pos + 1) =
sum(cost(twos(k1).two(k3):twos(k1).two(k3 + 1), pos + 1));
                                end
                                need(twos(k1).two(end), pos) =
sum(cost(twos(k1).two(end):numel(roads(k1).road), pos));
                                need(twos(k1).two(end), pos + 1) =
sum(cost(twos(k1).two(end):numel(roads(k1).road), pos));
                                end
                                end
                                end

                                now_fund = fund;

                                last_rate = 2;
                                for k2 = 1:numel(roads)
                                    pos = (k2 - 1) * 2 + 1;
                                    for k1 = 2:numel(roads(k2).road)
                                        % 剩余的 = 现有的减去消耗的

                                        if roads(k2).way(k1 - 1) == 2
                                            % 村庄补给
                                            % 第 k1 天第 road(k1)个区域的人数
                                            rate = 2;
                                            if position(k1, roads(k2).road(k1)) > 1
                                                rate = 4;
                                            end
                                            last_rate = rate;
                                            % k2 人数, k1 天数
                                            if water(k2) < need(k1, pos) && food(k2) < need(k1, pos + 1)
                                                water(k2) = water(k2) + need(k1, pos);
                                                food(k2) = food(k2) + need(k1, pos + 1);

                                                now_fund(k2) = now_fund(k2) - getM(need(k1, pos + 1), need(k1,
pos), rate);

```

```

        while getW( food(k2), water(k2) ) > bear_limit
            now_fund(k2) = now_fund(k2) + getM(1, 1, rate);
            water(k2) = water(k2) - 1;
            food(k2) = food(k2) - 1;
        end
    end

    if water(k2) < need(k1, pos)
        water(k2) = water(k2) + need(k1, pos);

        now_fund(k2) = now_fund(k2) - need(k1, pos) * rate *
water_basic_price;

        while getW( food(k2), water(k2) ) > bear_limit
            now_fund(k2) = now_fund(k2) + rate * water_basic_price;
            water(k2) = water(k2) - 1;
        end
    end

    if food(k2) < need(k1, pos + 1)
        food(k2) = food(k2) + need(k1, pos + 1);
        now_fund(k2) = now_fund(k2) - need(k1, pos + 1) * rate *
food_basic_price;

        while getW( food(k2), water(k2) ) > bear_limit
            food(k2) = food(k2) - 1;
            now_fund(k2) = now_fund(k2) + rate * food_basic_price;
        end
    end

elseif roads(k2).way(k1 - 1) == 1
    % 挖矿
    % k2 人数， k1 天数
    rate = 1; % 初始倍数是 1
    if position(k1, roads(k2).road(k1)) > 1
        % 如果多个人挖，倍数为 1 / 人数
        rate = 1 / position(k1, roads(k2).road(k1));
    end
end

```

```

        end
        now_fund(k2) = now_fund(k2) + rate * basic_earn;
    end

    % 结果矩阵有 3 列，要针对人数选择不同的结果列起点，一个人的结果是 1-3，两个人是 4-6，三个人是 7-9
    pos2 = (k2 - 1) * 3 + 1;
    spare(k1, pos2) = now_fund(k2);
    spare(k1, (pos2 + 1):(pos2 + 2)) = [water(k2), food(k2)] - cost(k1, pos:(pos + 1));

    water(k2) = water(k2) - cost(k1, pos);
    food(k2) = food(k2) - cost(k1, pos + 1);

    ak = 0; % 水弥补数
    bk = 0; % 食物弥补数
    if water(k2) < 0
        ak = -water(k2);
    end
    if food(k2) < 0
        bk = -food(k2);
    end
    if ak > 0 || bk > 0
        for kk = k1:-1:1
            spare(kk, pos2 + 1) = spare(kk, pos2 + 1) + ak;
            spare(kk, pos2 + 2) = spare(kk, pos2 + 2) + bk;
            spare(kk, pos2) = spare(kk, pos2) - getM(bk, ak, last_rate);
            if ismember(kk, twos(k2).two)
                break;
            end
        end
    end
end
end
end

```

```

% 出现多余的时候
% 在终点按半价换钱
for k1 = 1:numel(roads)

    po2 = (k1 - 1) * 3 + 1;
    endNum = numel(roads(k1).road);

    w1 = spare(endNum, po2 + 1); % 水多余量
    f1 = spare(endNum, po2 + 2); % 食物多余量

    spare(endNum, po2 + 1) = spare(endNum, po2 + 1) - w1;
    spare(endNum, po2 + 2) = spare(endNum, po2 + 2) - f1;
    spare(endNum, po2) = spare(endNum, po2) + getM(f1, w1, 0.5);

end

size1 = numel(roads(1).road);
size2 = numel(roads(2).road);
size3 = numel(roads(3).road);
data = zeros(max([size1, size2, size3]), 15);
data(1:size1, 1:5) = [[0:size1-1]' roads(1).road' spare(1:size1, 1:3) ];
data(1:size2, 6:10) = [[[0:size2-1]' roads(2).road' spare(1:size2, 4:6) ]];
data(1:size3, 11:15) = [[[0:size3-1]' roads(3).road' spare(1:size3, 7:9) ]];
return ;
end

function [w] = getW(food, water)
    global water_weight;
    global food_weight;
    w = water * water_weight + food * food_weight;
end

function [m] = getM(food, water, c)
    global water_basic_price;

```

```

    global food_basic_price;

    m = water * c * water_basic_price + food * c * food_basic_price;
end

function [] = init_info(add)

    global bear_limit;      % 负重上限
    global deadline;        % 截止日期
    global basic_fund;      % 基础资金
    global basic_earn;      % 基础收益
    global water_weight;    % 水每箱质量
    global food_weight;     % 食物每箱质量
    global water_basic_price; % 水基准价
    global water_consum;    % 1: 水晴朗消耗 2: 水高温消耗 3: 水沙暴消耗
    global food_basic_price; % 食物基准价
    global food_consum;     % 1: 食物晴朗消耗 2: 食物高温消耗 3: 食物沙暴消耗
    global weather; % 天气情况

%    % 晴朗极端天气
%    weather_info = readmatrix(add, 'Range', 'A16:J21', 'OutputType', 'string');

%    % 高温极端天气
%    weather_info = readmatrix(add, 'Range', 'A28:J33', 'OutputType', 'string');
%
% 一般天气
weather_info = readmatrix(add, 'Sheet', '第六关', 'Range', 'A11:J16', 'OutputType',
'string');

weather = zeros(30, 1);
for k1 = 1:2:5
    for k2 = 1:10
        day = str2double(weather_info(k1, k2));
        winfo = weather_info(k1 + 1, k2);
        val = -1;
        switch( winfo )
            case "晴朗"

```

```

        val = 1;
    case "高温"
        val = 2;
    case "沙暴"
        val = 3;
    end
    weather(day) = val;
end
end

bear_limit_text = char(readmatrix(add, 'Sheet', '第六关', 'Range', 'C3:C3', 'OutputType',
'string')); % 负重上限信息

deadline_text = char(readmatrix(add, 'Sheet', '第六关', 'Range', 'C4:C4', 'OutputType',
'string')); % 截止日期信息

basic_fund_text = char(readmatrix(add, 'Sheet', '第六关', 'Range', 'E3:E3', 'OutputType',
'string')); % 基础资金信息

basic_earn_text = char(readmatrix(add, 'Sheet', '第六关', 'Range', 'E4:E4', 'OutputType',
'string')); % 基础收益信息


bear_limit = str2double( bear_limit_text( 1:( strfind( bear_limit_text, '千' ) - 1 ) ) );
deadline   = str2double( deadline_text( ( strfind( deadline_text, '第' ) +
1 ):( strfind( deadline_text, '天' ) - 1 ) ) );
basic_fund = str2double( basic_fund_text( 1:( strfind( basic_fund_text, '元' ) - 1 ) ) );
basic_earn = str2double( basic_earn_text( 1:( strfind( basic_earn_text, '元' ) - 1 ) ) );


water_consum = zeros(3, 1);
food_consum = zeros(3, 1);


arr = num2cell(readmatrix(add, 'Sheet', '第六关', 'Range', 'B8:F8'));
[water_weight,      water_basic_price,      water_consum(1),      water_consum(2),
water_consum(3)] = arr{:};


arr = num2cell(readmatrix(add, 'Sheet', '第六关', 'Range', 'B9:F9'));
[food_weight, food_basic_price, food_consum(1), food_consum(2), food_consum(3)] =
arr{:};

```

```
clear weather_info day winfo val bear_limit_text deadline_text basic_fund basic_earn  
arr dis;  
return ;  
end
```